Mensajería

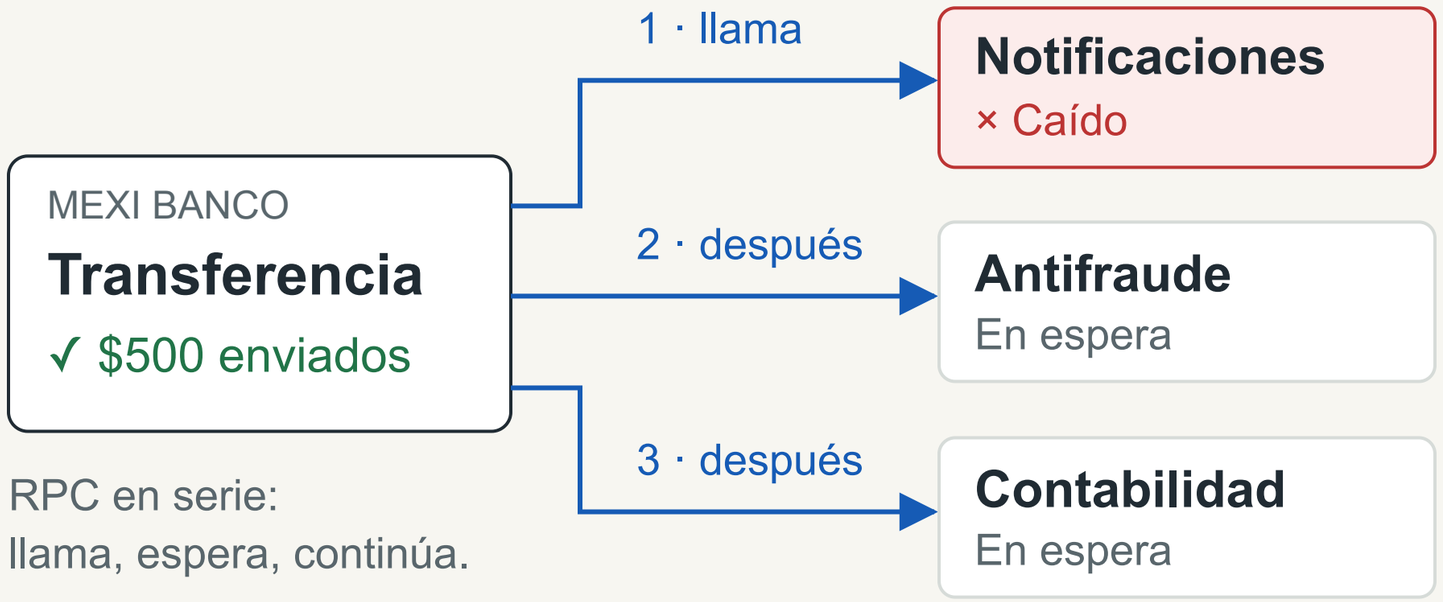
Que se entere quien deba

LA PREGUNTA DE HOY

¿Cómo se entera todo el sistema de que algo pasó, sin que quien lo cuenta sepa quién escucha?

Falla la solicitud
por un servicio secundario.

HOY / TRES LLAMADAS, UNA DEPENDENCIA



RPC en serie:
llama, espera, continúa.

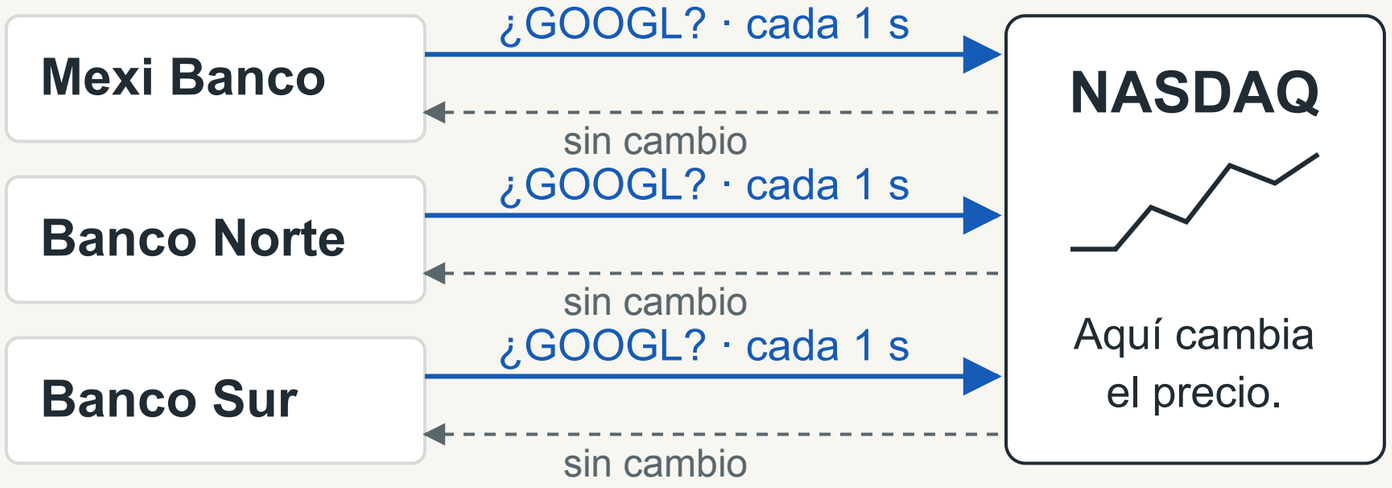
Enviar dinero no debería depender de enviar un aviso.

Polling

¿Ya cambió? ¿Y ahora?

Preguntar cada cierto tiempo si algo cambió. Funciona, pero casi todas las respuestas regresan vacías.

Miles de símbolos · Decenas de millones de operaciones al día · Cada banco pregunta por todos

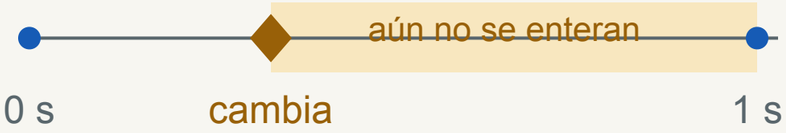


Un símbolo ilustrado. Repite esto por todos.

01 · Desperdicio

Trabajo para decir «sin cambio».

02 · Retraso



03 · Carga

Más frecuencia → más consultas
→ riesgo de saturar al NASDAQ.

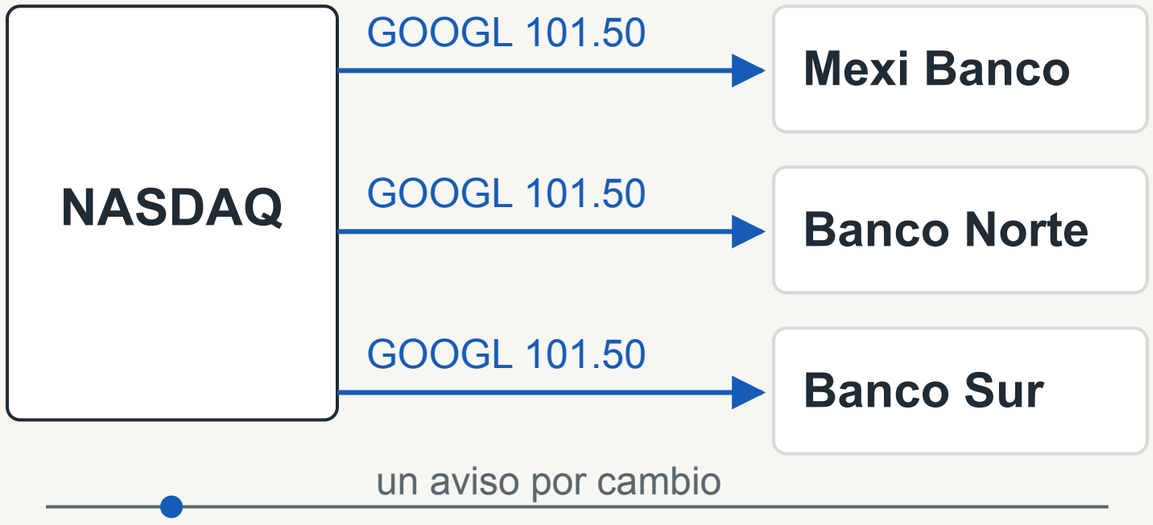
Quien sabe del cambio es quien tiene que avisar.

Push

¿Y si el NASDAQ avisa?

Quien tiene el dato avisa cuando cambia. Los interesados dejan de preguntar.

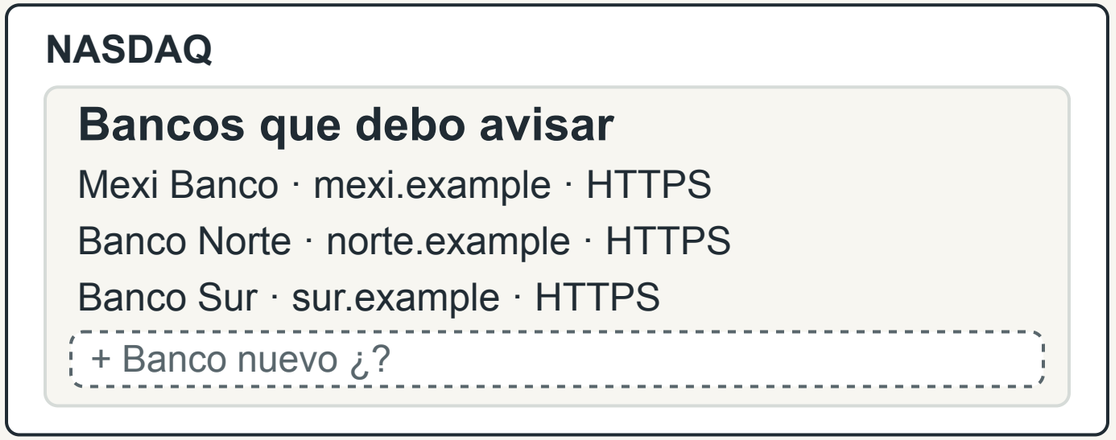
LO QUE SE GANA



✓ sin preguntas vacías

✓ sin retraso

EL PROBLEMA NUEVO



✗ El NASDAQ conoce a cada banco

✗ Banco nuevo = cambiar el código del NASDAQ

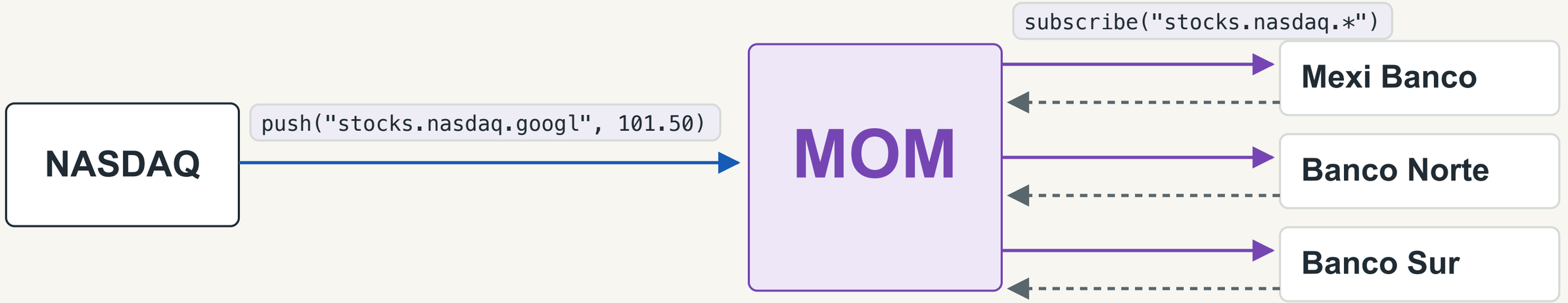
✗ Banco caído = ¿quién guarda el aviso?

Avisar arregla el polling, pero ahora el emisor tiene que conocer a todos.

MOM

¿Y si ponemos una caja en medio?

Message Oriented Middleware: un intermediario que recibe los mensajes de quien los produce y los entrega a quien se interesó en ellos. Emisor y receptores solo conocen al MOM.



- Canal: un nombre, con comodines `*`
- El banco se suscribe una vez.
- El NASDAQ publica sin saber quién escucha.
- Banco nuevo = una suscripción más; el NASDAQ no cambia.

Suscripciones

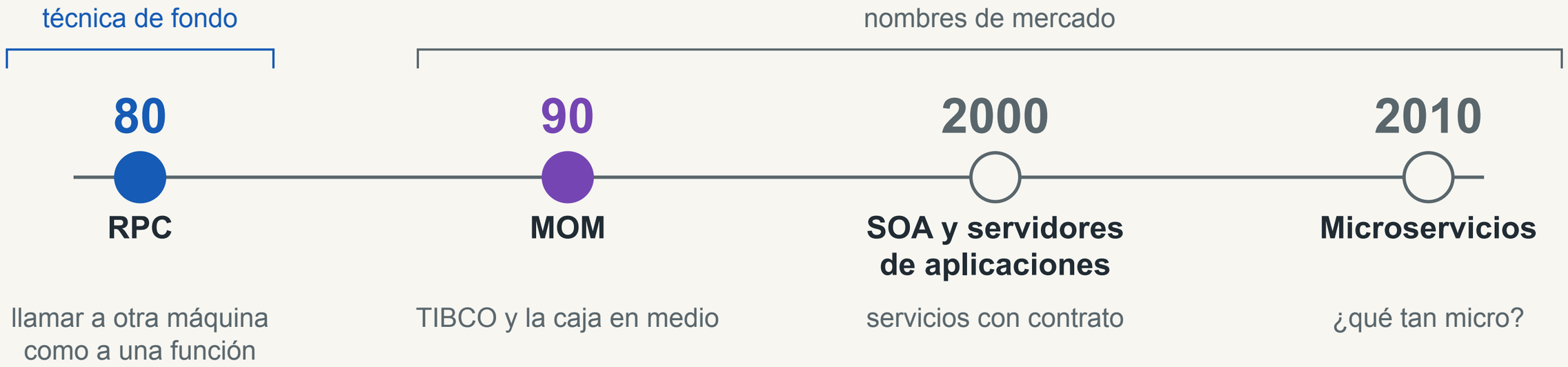
Canal	Suscriptor
`stocks.nasdaq.*`	Mexi Banco
`stocks.nasdaq.*`	Banco Norte
`stocks.nasdaq.googl`	Banco Sur

El emisor ya no conoce a nadie: solo conoce al MOM.

Historia

¿Esto es nuevo?

No. Cada década cambió el nombre de moda; debajo siguen el RPC y la mensajería.



La mensajería se sigue usando: junto con una base de datos, suele ser lo primero con que se arma un sistema distribuido.

Las modas cambian de nombre; el RPC y la mensajería se quedan.

Mensaje

¿Qué viaja por el MOM?

Un paquete chico de datos con un encabezado, que lee el broker, y un cuerpo, que lee el consumidor. Viaja como bytes.

ENCABEZADO

```
id: "5d2f-81a0"
tipo: "TransferenciaRealizada"
origen: "transferencias"
fecha: "2026-10-17T10:42:07Z"
```

lo lee el broker

CUERPO

```
{ "cuenta": "MX-1001",
"destino": "MX-2002",
"monto": 500.00,
"moneda": "MXN" }
```

lo lee el consumidor

se serializa: JSON → bytes

7b	22	63	75	65	6e	74	61	22	…

Chico

Kilobytes, no un video: si pesa, manda la ruta del archivo.

Autocontenido

Trae todo lo que el consumidor necesita.

`{ "id": 81 }` ✗ obliga a llamar de vuelta

✓ el cuerpo de la izquierda

Serializado

Viaja como bytes: `JSON` `Avro` `Protobuf`

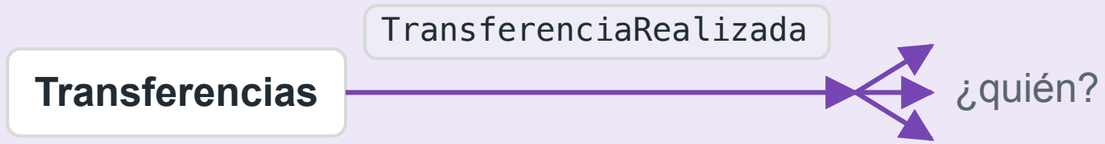
Un mensaje es un hecho empaquetado: no obliga a llamar de vuelta.

Evento o comando

¿El mensaje cuenta algo o pide algo?

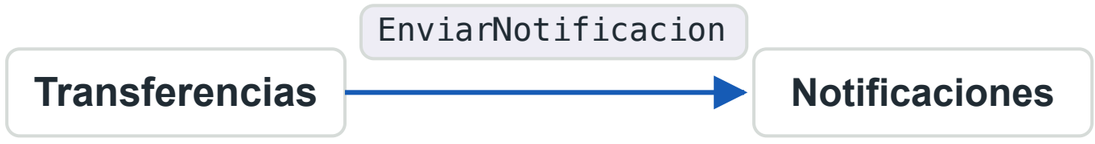
Un evento avisa un hecho que ya ocurrió. Un comando le pide a un destinatario que haga algo.

EVENTO



Ya pasó. Que se entere quien quiera.

COMANDO



Haz esto. Te lo pido a ti.

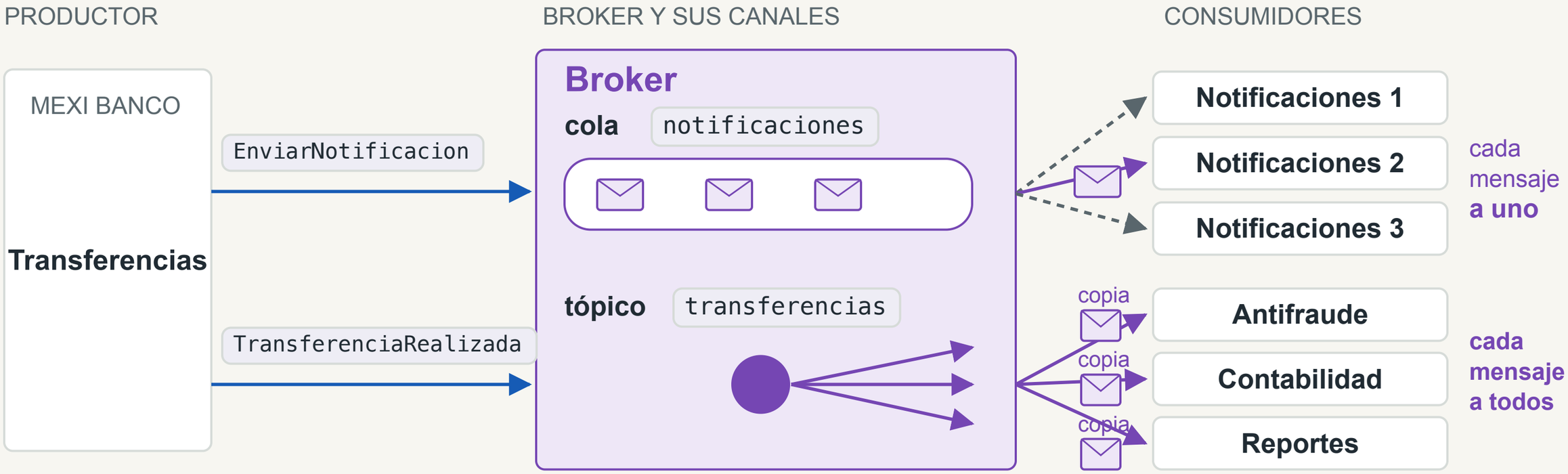
	Evento	Comando
Nombre	Verbo en pasado: `TransferenciaRealizada`	Imperativo: `EnviarNotificacion`
Destinatarios	Cero, uno o muchos; el emisor no sabe cuántos	Uno, y el emisor sabe cuál
¿Se puede rechazar?	No: ya ocurrió	Sí: puede fallar o negarse
En el NASDAQ	`PrecioActualizado(GOOGL, 101.50)`	No aplica: el NASDAQ no le ordena nada a los bancos

Un evento cuenta lo que pasó; un comando pide que pase algo.

Roles

¿Quién manda, quién recibe y por dónde?

El productor publica en un canal con nombre; el broker guarda y entrega; el consumidor recibe. El canal es una cola o un tópico.

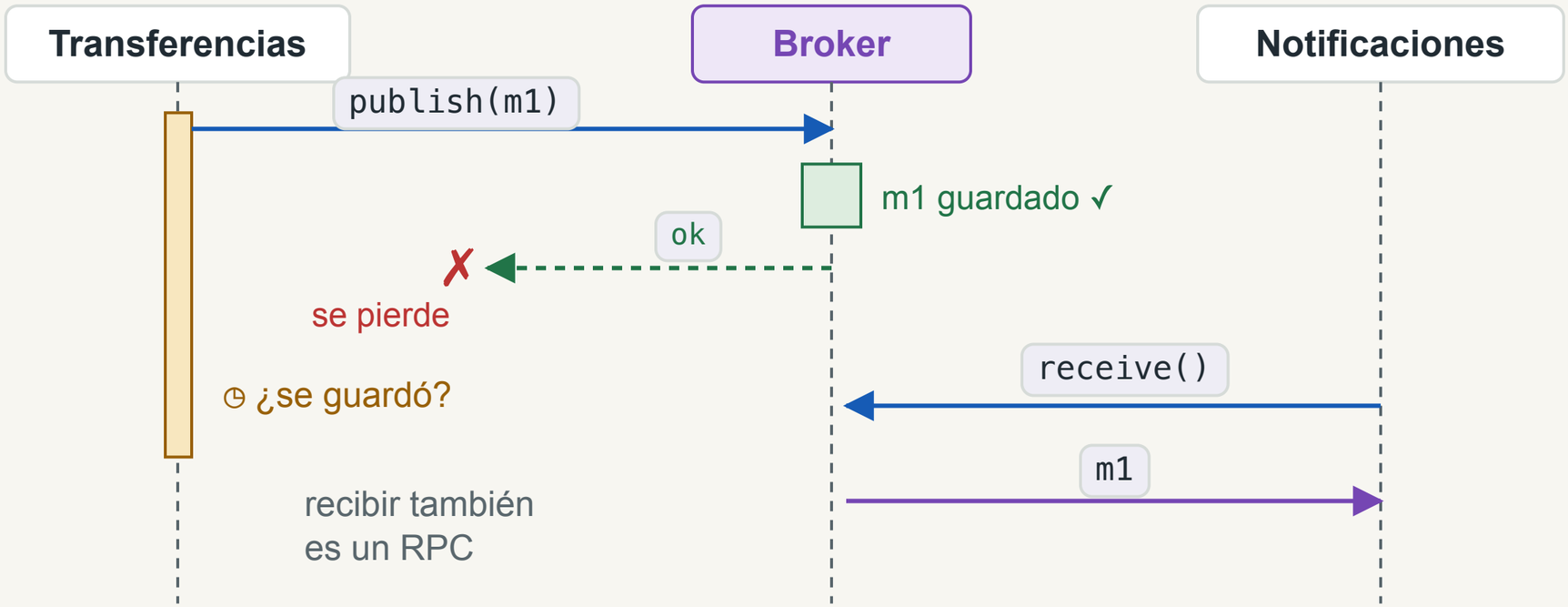


Productor y consumidor no se conocen: solo comparten el nombre del canal.

Sigue siendo RPC

¿El productor ya no espera a nadie?

Publicar es un RPC al broker y recibir es otro RPC. El productor ya no espera al consumidor, pero sí al broker, y esa respuesta también se puede perder.



Transferencias solo puede elegir:

Reintentar

m1 puede quedar dos veces en el broker

No reintentar

si no se guardó, m1 se perdió

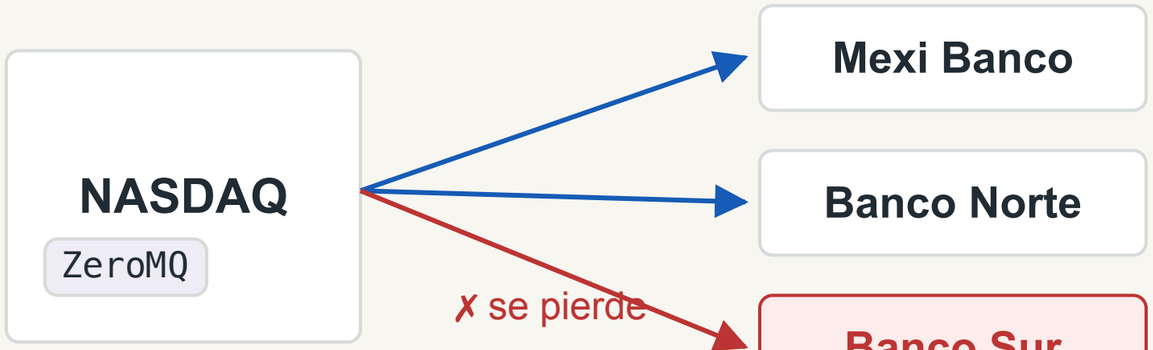
El MOM no elimina el RPC: ahora el productor duda del broker.

Directa o broker

¿Hace falta un servidor en medio?

En la mensajería directa una biblioteca simula la cola y los nodos se hablan entre sí.
Con un broker, un servidor dedicado recibe, guarda y reparte.

MENSAJERÍA DIRECTA

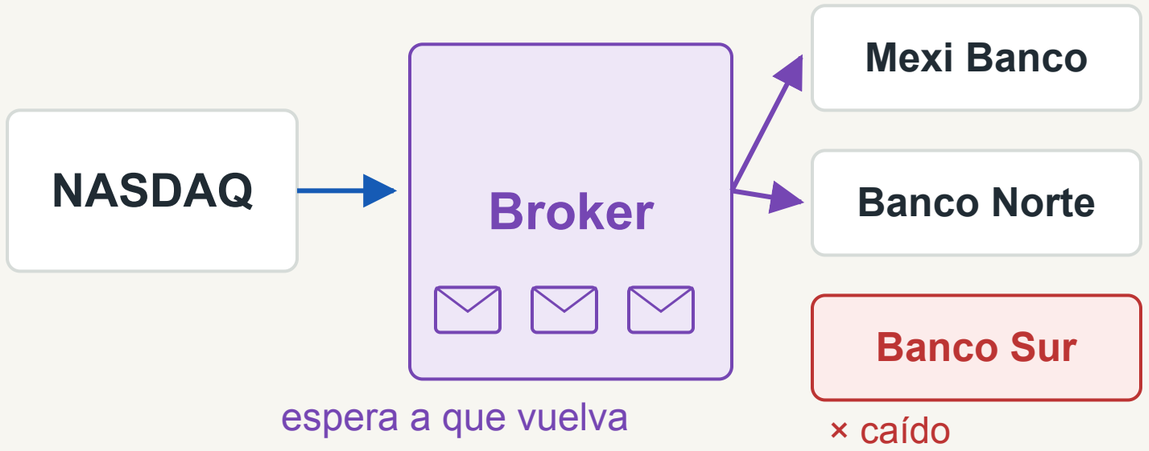


biblioteca, no servidor

`ZeroMQ` `webhooks`

✓ sin pieza extra, un salto menos
✗ todos tienen que estar en línea

BROKER



espera a que vuelva

`RabbitMQ` `SQS` `Google Pub/Sub`

✓ guarda mientras el receptor no está
✗ una pieza más que operar y vigilar

Sin broker nadie guarda el mensaje: todos tienen que estar en línea al mismo tiempo.

Broker

Si guarda mensajes, ¿no es una base de datos?

Sí: una base de datos especializada en mensajes. Guarda poco, por poco tiempo, y avisa en lugar de esperar consultas.

		 **Base de datos** (Postgres)	 **Broker** (RabbitMQ, SQS)
se parecen	**Sobrevive a un reinicio**	✓ guarda en disco	✓ guarda en disco cola `durable` mensaje `persistent`
difieren	**Al leer el dato**	Se queda	Se borra cuando el consumidor confirma
difieren	**Cuánto guarda**	Todo, por años	Poco: colas cortas
difieren	**Cómo se busca**	Consultas e índices	Por nombre de canal, sin índices
difieren	**Si el dato cambia**	Tú preguntas	Te avisa

Excepción: en Kafka leer no borra. Lo vemos en event streaming.

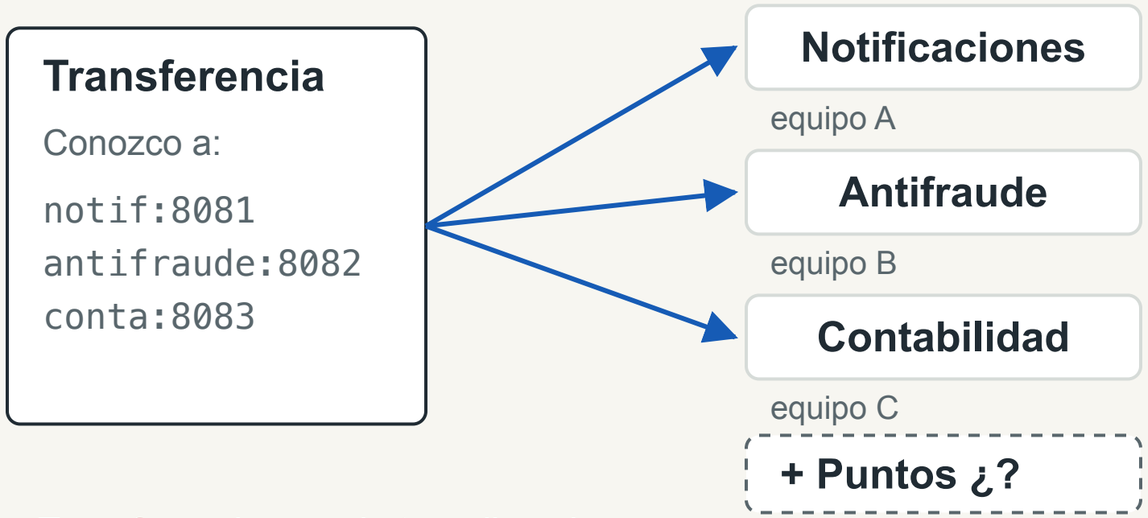
Un broker es una base de datos que olvida a propósito.

Espacio

¿Tengo que saber quién me escucha?

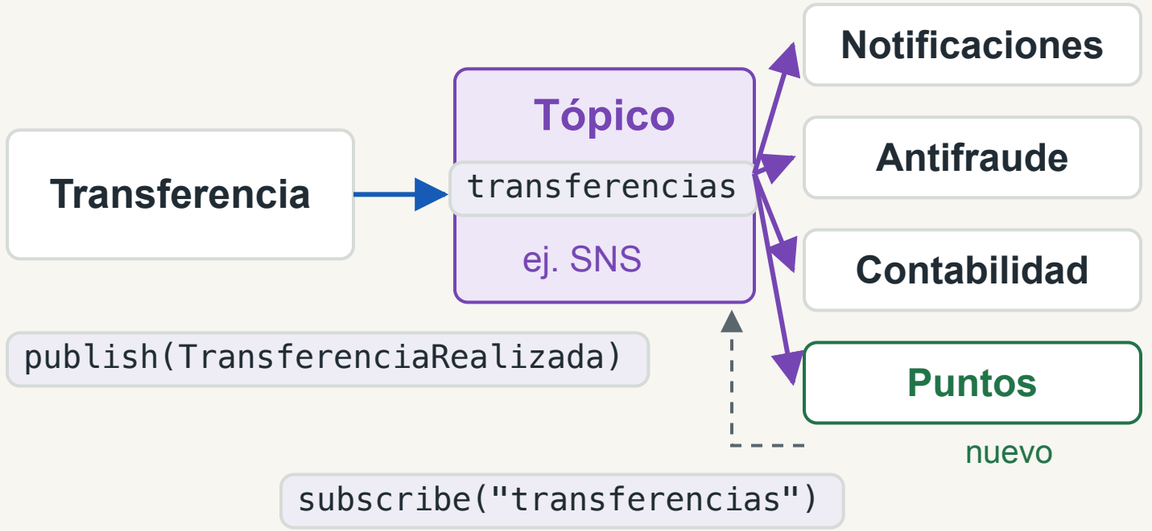
Desacople en espacio: el emisor publica en un canal y el broker sabe a quién entregarlo.
El emisor no guarda direcciones, y su equipo no se coordina con cada equipo que escucha.

CON RPC



✗ Transferencia guarda tres direcciones
✗ Servicio nuevo = cambiar Transferencia y agendar con su equipo

CON BROKER



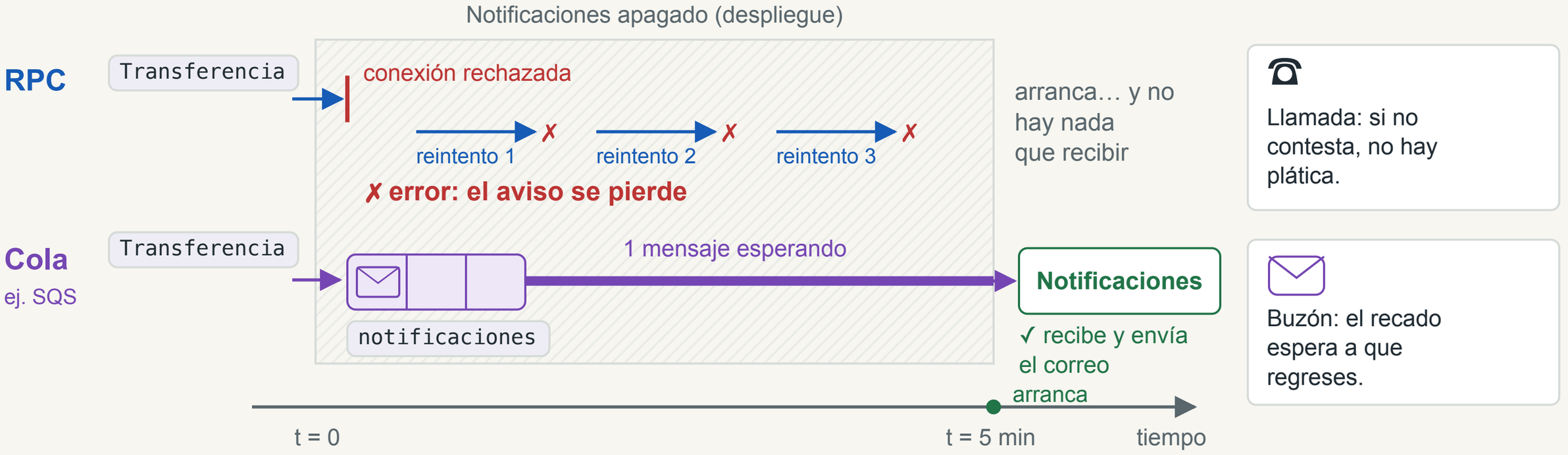
✓ Servicio nuevo = una suscripción más
✓ Transferencia no cambia ni se entera

Quien publica no sabe quién escucha, ni tiene que pedirle permiso a nadie.

Tiempo

¿Y si el receptor está apagado cuando aviso?

Desacople en tiempo: el receptor no tiene que estar vivo cuando sale el mensaje.
El mensaje espera guardado en el broker hasta que el receptor se conecta.

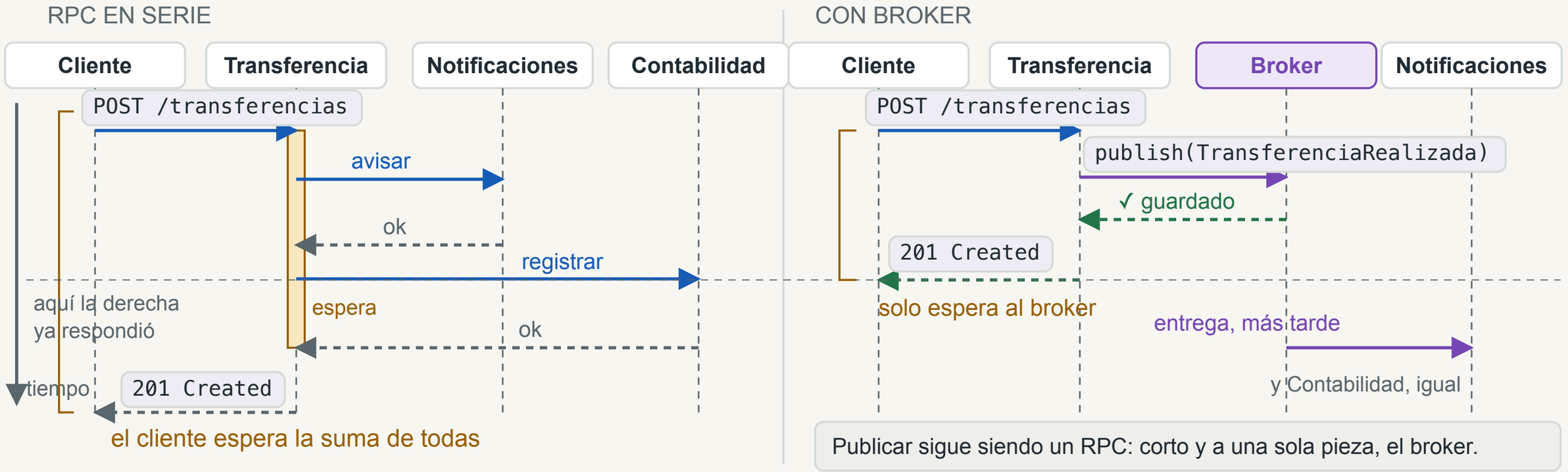


El mensaje espera al receptor: ya no tienen que estar vivos al mismo tiempo.

Sincronía

¿Tengo que quedarme esperando la respuesta?

Desacople en sincronía: el emisor entrega el mensaje y sigue con lo suyo.
No espera a que el receptor lo procese; solo espera a que el broker lo guarde.

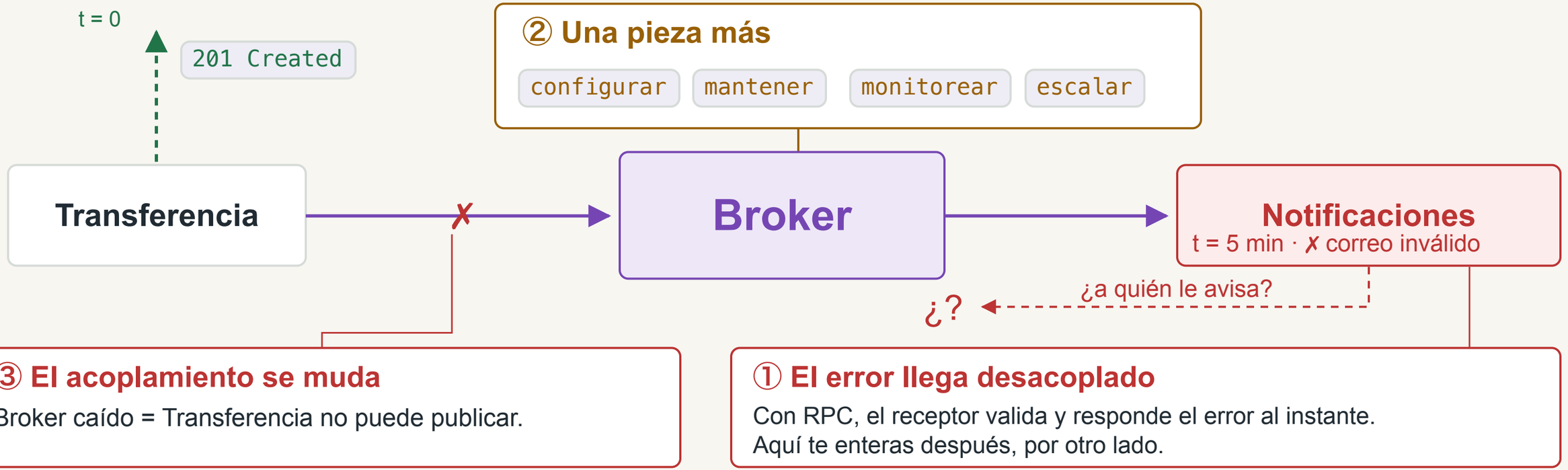


La transferencia responde cuando el dinero se movió, no cuando todos se enteraron.

El costo

¿Qué pagamos por desacoplar?

Cada desacople tiene su precio: el error llega después y por otro lado,
hay una pieza más que operar, y el acoplamiento se muda al broker.

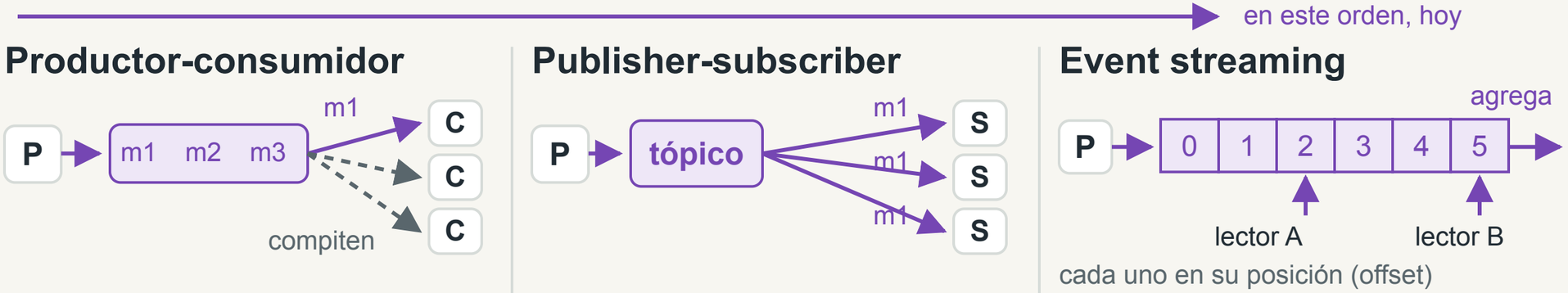


Desacoplar no quita las fallas: las cambia de momento y de lugar.

Tres modelos

¿Quién recibe cada mensaje y qué pasa con él después?

Todo broker cae en uno de tres modelos: los distingue a quién llega cada mensaje y si se borra al leerlo.



	Productor-consumidor	Publisher-subscriber	Event streaming
Ejemplo	`SQS`	`SNS`	`Kafka`
¿Quién recibe cada mensaje?	Uno de los consumidores	Todos los suscriptores	Todos los que lean, cada quien a su ritmo
¿Se borra al leerlo?	Sí, cuando el consumidor confirma	Sí, al entregarse	**No: lo borra la retención**
Lo usas cuando	hay trabajo pesado que repartir	varios servicios distintos deben enterarse	necesitas orden o releer el pasado

Uno lo recibe, todos lo reciben, o se queda escrito: el resto de la clase es esta tabla.

Trabajo en picos

¿Quién genera todos los estados de cuenta el día de corte?

Hay trabajo pesado (segundos de CPU por tarea) que llega de golpe. Si se hace en el momento, o sobra capacidad casi todo el mes o falta el día que importa.

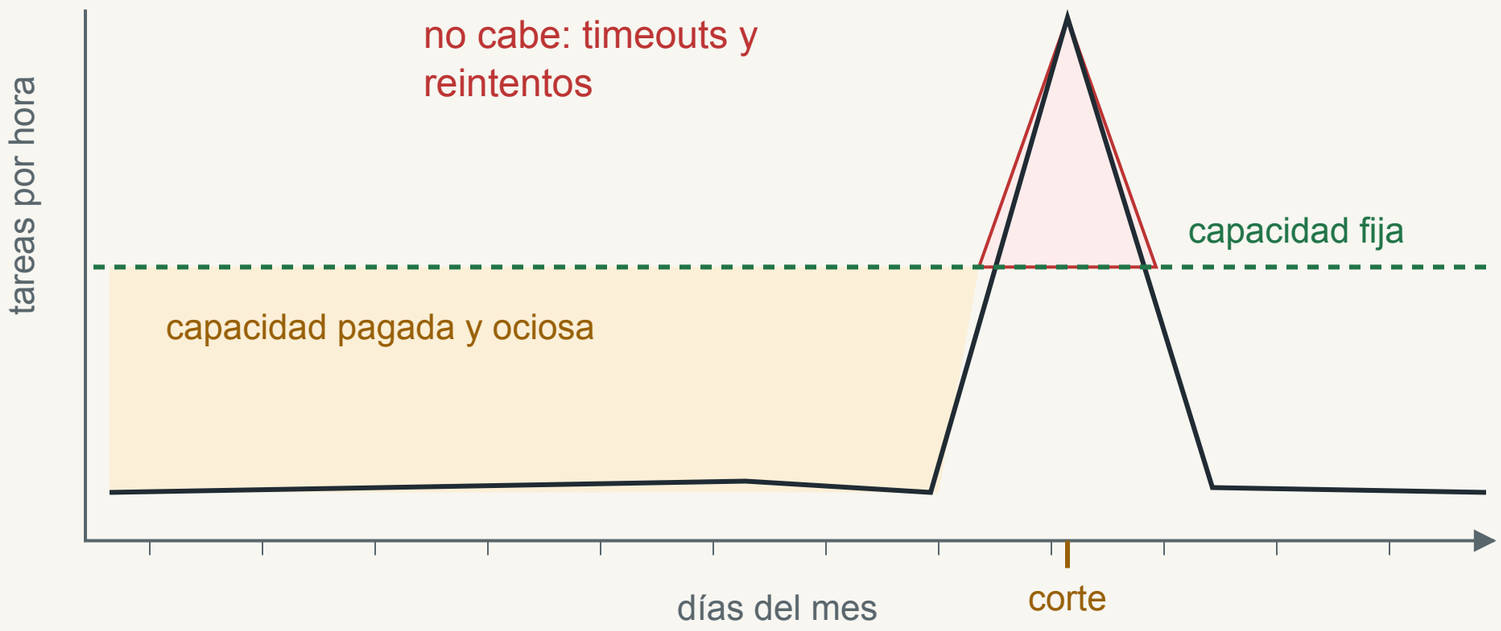
POR CADA CUENTA, EL DÍA DE CORTE

- 1 · generar el PDF del estado
- 2 · guardarlo
- 3 · abonar el cashback del periodo

El mismo patrón en otro lado:

`transcodificar video`

todos suben videos en el Mundial

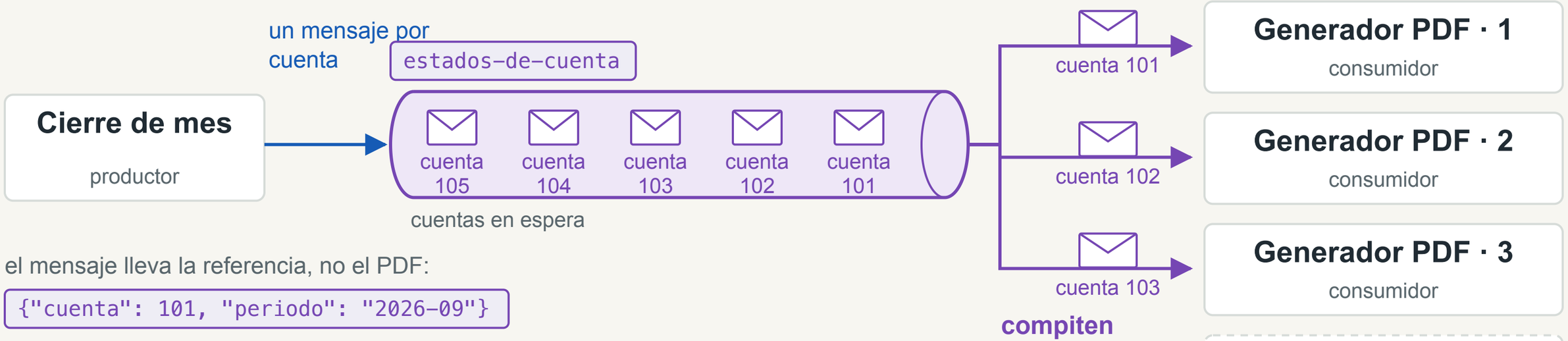


El trabajo llega en picos; la capacidad, no.

Productor-consumidor

¿Y si el trabajo se forma en una fila?

El productor deja mensajes en una cola y varios consumidores compiten por ellos. Cada mensaje lo procesa un solo consumidor.



el mensaje lleva la referencia, no el PDF:

```
{"cuenta": 101, "periodo": "2026-09"}
```

Compiten: uno por mensaje; balanceo de carga.
Se borra cuando un consumidor lo procesa.

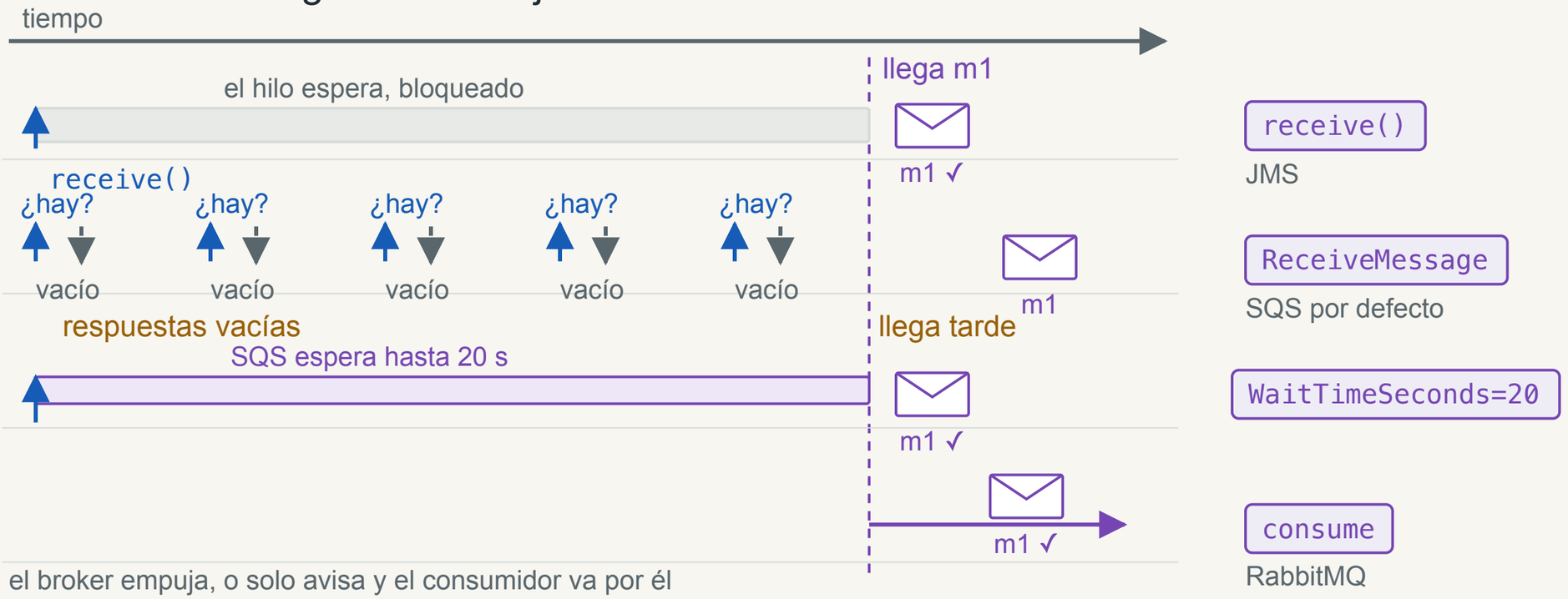
No necesariamente FIFO: una gran bolsa.
Escala sin cambiar el productor.

La cola absorbe el pico; los consumidores lo vacían a su ritmo.

Formas de recibir

¿Cómo se entera el consumidor de que hay trabajo?

Enviar a la cola casi siempre es push. Recibir tiene cuatro variantes; cambian cuánto se pregunta en vano y cuánto tarda en llegar el mensaje.

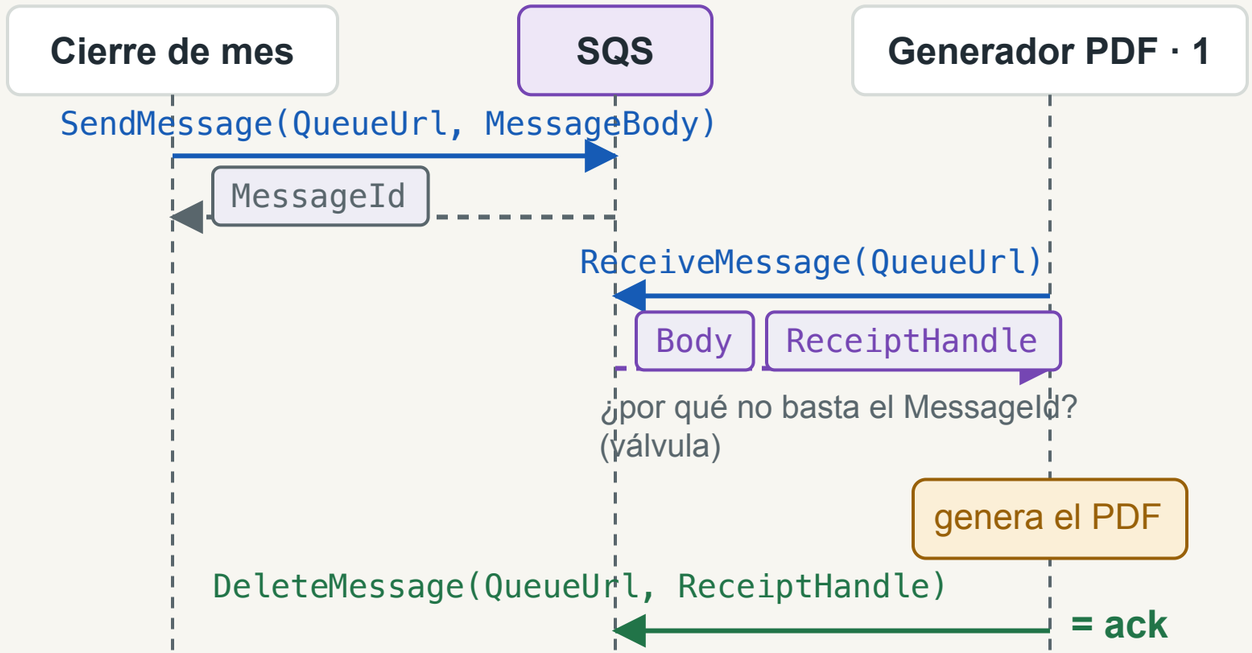


Long polling: preguntar una vez y que la cola espere por ti.

API de SQS

¿Cómo se habla con la cola?

Tres operaciones: enviar, recibir y borrar. Cada una es un RPC a la cola.



consumidor.py · boto3

```
1  while True:
2  r = sqs.receive_message(QueueUrl=url,
3  WaitTimeSeconds=20)
4  for m in r.get("Messages", []):
5  procesar(m) # genera el PDF
6  sqs.delete_message(QueueUrl=url,
7  ReceiptHandle=m["ReceiptHandle"])
```

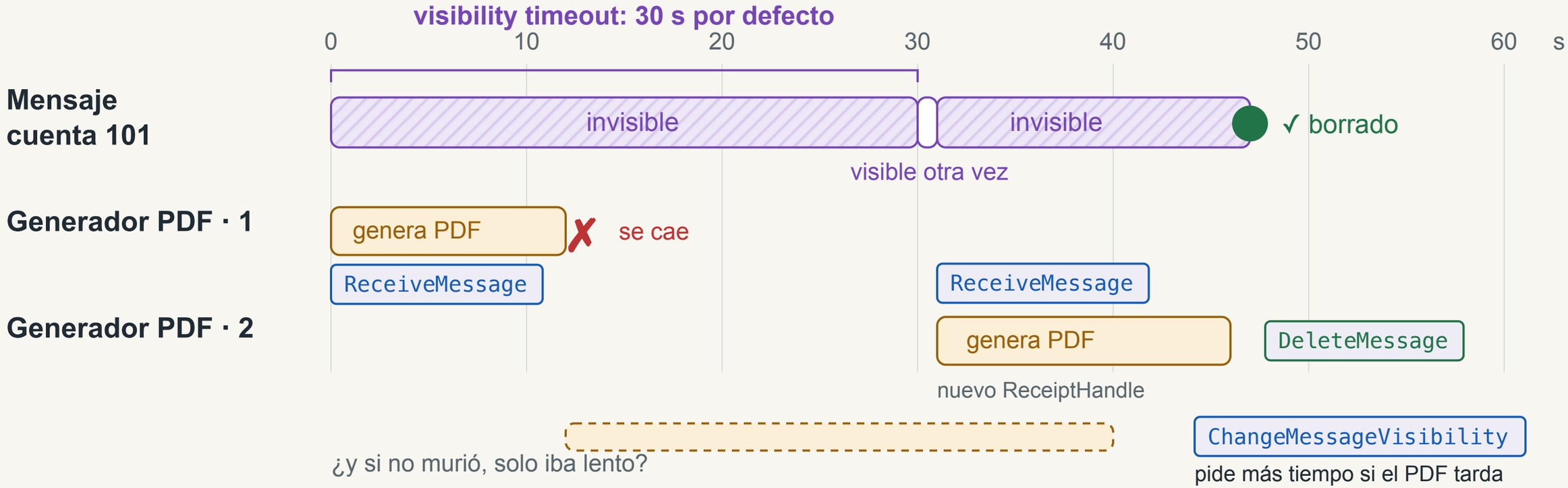
primero procesar, después borrar

Recibir no borra: el consumidor borra cuando terminó.

Visibility timeout

¿Y si el consumidor muere a la mitad?

Al entregar, SQS no borra el mensaje: lo esconde un rato. Si nadie lo borra antes de que venza, vuelve a ser visible para otro consumidor.

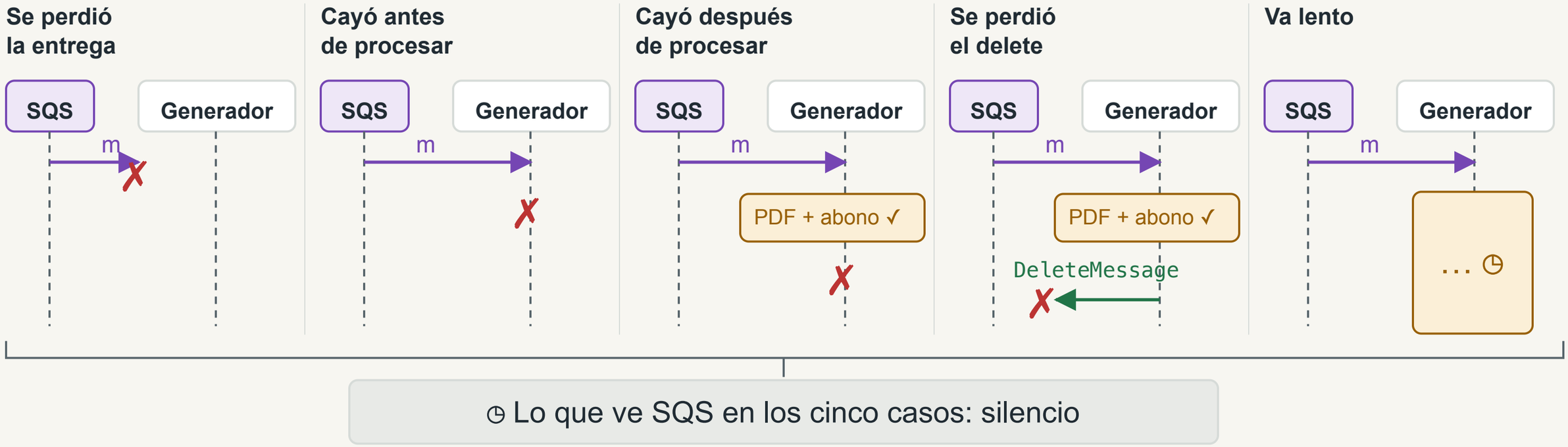


Esconder en vez de borrar: si el consumidor muere, el mensaje regresa solo.

Emisor incierto

¿Por qué SQS no sabe si el PDF se generó?

Después de entregar y antes del DeleteMessage, SQS solo ve silencio. Cinco situaciones distintas se ven exactamente igual desde su lado.



Sin respuesta, el emisor no distingue "no pasó" de "ya pasó".

At-most o at-least

¿Borro antes o después de procesar?

Ante la duda solo hay dos salidas: rendirse (at-most-once, 0 o 1 veces) o reintentar (at-least-once, 1 o más). Ningún orden logra exactamente una.

Anotar y luego procesar

① `DeleteMessage`

② ✗ el generador se cae

③ ~~procesar(m)~~ nunca ocurre

0 veces `at-most-once`

rendirse: el estado nunca se genera

Procesar y luego anotar

① procesar(m) · PDF + abono

② ✗ el generador se cae

③ ~~DeleteMessage~~ · nunca llega

↓ vence el timeout → otro generador

④ procesar(m) · otra vez

2 veces `at-least-once`

reintentar: abono doble

SQS estándar elige este

No hay orden que salve: se escoge entre perder o repetir.

Consumidor idempotente

¿Y si procesar dos veces diera lo mismo que una?

Idempotente: f(f(x)) = f(x). Una llave por tarea, igual en cada reintento, y registrar y procesar en una sola transacción.

procesar.py

```
1  def procesar(m):
2  c = json.loads(m["Body"])
3  # {"llave": "101:2026-09", ...}
4  guardar_pdf(c) # misma ruta: idempotente natural
5  try:
6  with db.transaction():
7  db.insert("procesados", llave=c["llave"]) # UNIQUE
8  db.abonar(c["cuenta"], c["cashback"])
9  except UniqueViolation: pass # ya estaba: no se repite
```

una transacción

¿Idempotente de natural?

efecto	¿repetir hace daño?
`guardar estados/101/2026-09.pdf`	✓ no: sobrescribe el mismo archivo
`abonar cashback`	✗ sí: cada vez suma

at-least-once + idempotencia
= exactly-once semántico

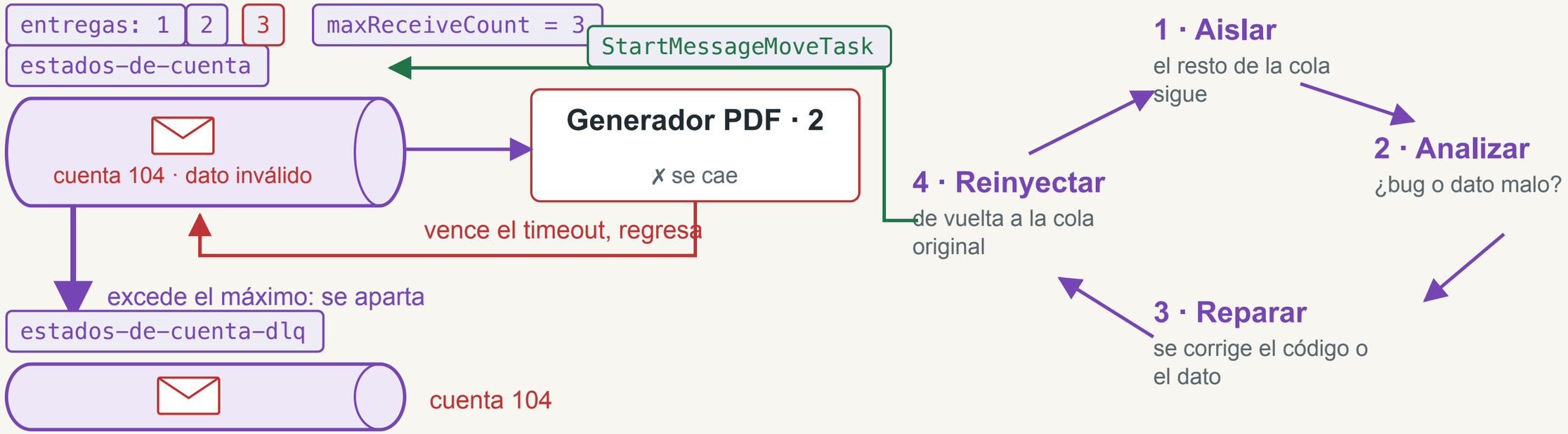
equivalente, no igual:
dos PDF iguales, un solo abono

El duplicado no se puede evitar; se puede volver inofensivo.

Dead letter queue

¿Y si un mensaje tumba a cada consumidor que lo toma?

Un poison pill falla siempre y regresa después de cada timeout. Un contador de entregas lo aparta a una cola aparte: la DLQ.



Una DLQ sin monitoreo es un cementerio: alarma en cuanto tenga mensajes.

Al mensaje que falla siempre se le aparta, no se le reintenta para siempre.

Cola llena

¿Y si los consumidores no alcanzan a vaciarla?

Una cola absorbe picos, no un desbalance permanente. Si entra más de lo que sale, crece hasta que algo cede.

Cola con límite

(un broker en tu servidor)



`rechazar`

el productor decide qué hacer

`descartar`

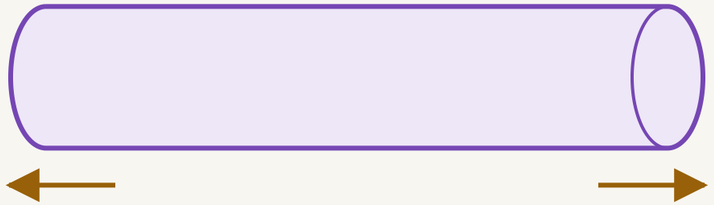
se pierden mensajes

`frenar al productor`

backpressure

En la nube

(SQS)



$ escala sola, y se paga

✗ lo más viejo que la retención se borra (4 días por defecto)

Monitorear

`ApproximateNumberOfMessagesVisible`

cuántos esperan

`ApproximateAgeOfOldestMessage`

qué tan viejo es el más viejo

`ApproximateNumberOfMessagesNotVisible`

cuántos están en proceso

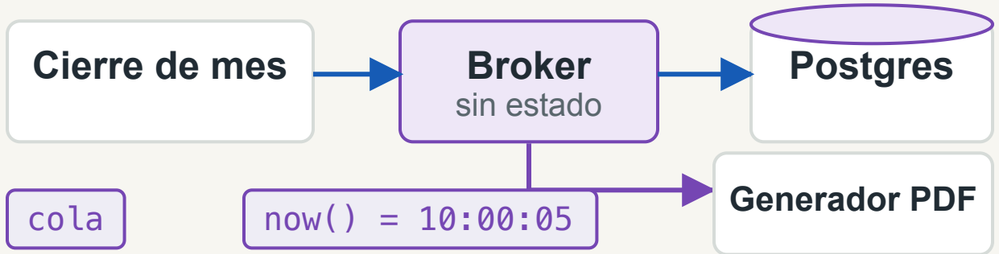
→ más consumidores

Una cola que solo crece no es un buffer: es una falla con retraso.

Cola en Postgres

¿Cómo harías SQS con lo que ya sabes?

Un broker sin estado frente a una tabla. El visibility timeout es solo una columna de fecha.



id	mensaje	visible_desde	
41	cuenta 101	10:00:00	visible
42	cuenta 102	10:00:30	escondido
43	cuenta 103	10:00:31	escondido
44	cuenta 104	09:59:58	visible

cola.sql · PostgreSQL

```
1  INSERT INTO cola (mensaje, visible_desde)
2  VALUES ($1, now());
3  UPDATE cola
4  SET visible_desde = now() + interval '30 seconds'
5  WHERE id = (SELECT id FROM cola
6  WHERE visible_desde <= now()
7  ORDER BY id LIMIT 1
8  FOR UPDATE SKIP LOCKED)
9  RETURNING id, mensaje;
10 DELETE FROM cola WHERE id = $1;
```

SendMessage: 1 a 2 · ReceiveMessage: 3 a 9 · DeleteMessage: 10

✓ si cae el broker, no pasa nada

✗ si cae Postgres, se pierde la cola

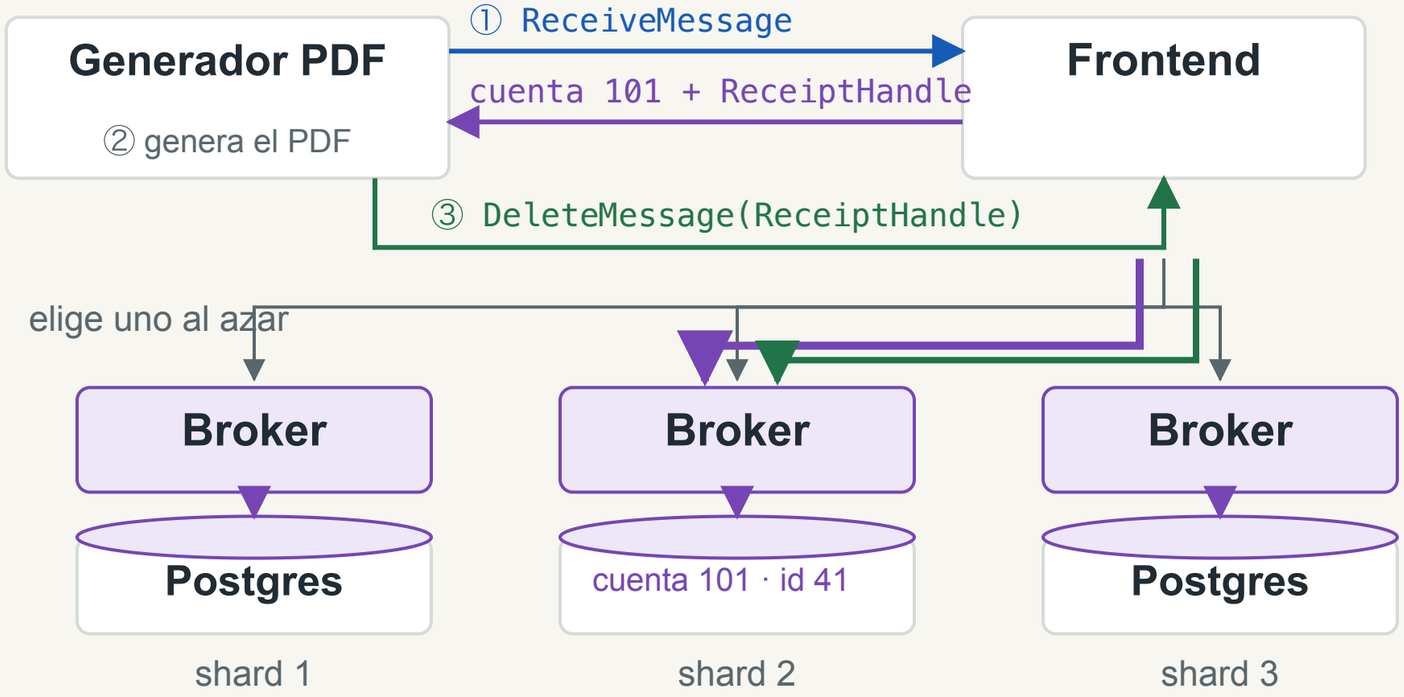
✗ no escala: una sola base

Un broker es una base de datos que borra lo que entrega.

ReceiptHandle

¿Por qué SQS no borra con el MessageId?

Para escalar, muchos pares broker y base: se escribe en uno al azar y se lee de uno al azar. El delete tiene que saber a cuál volver.



Qué lleva el ReceiptHandle

`cifrar(shard 2, id 41)`

opaco para el cliente: solo lo devuelve

Falso vacío

si el receive cae en un shard sin mensajes, regresa vacío aunque haya en otro

`WaitTimeSeconds > 0`

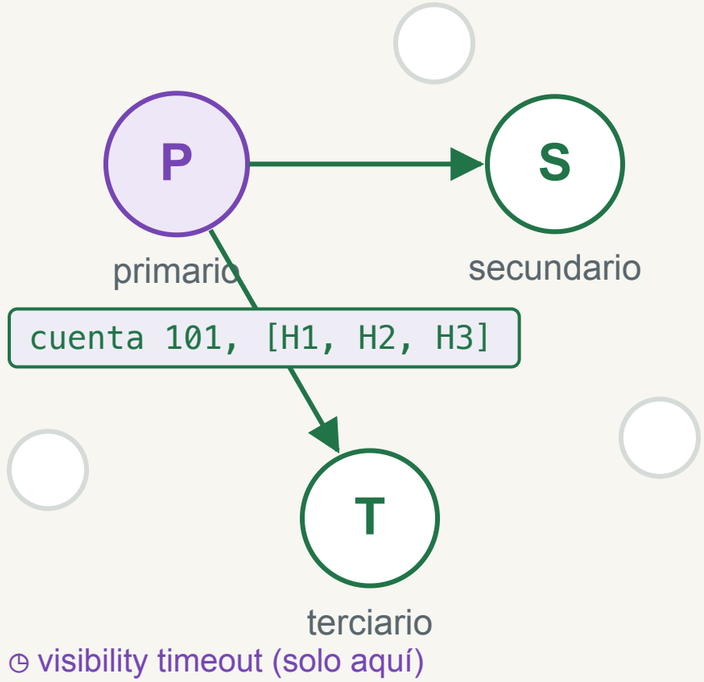
consulta todos

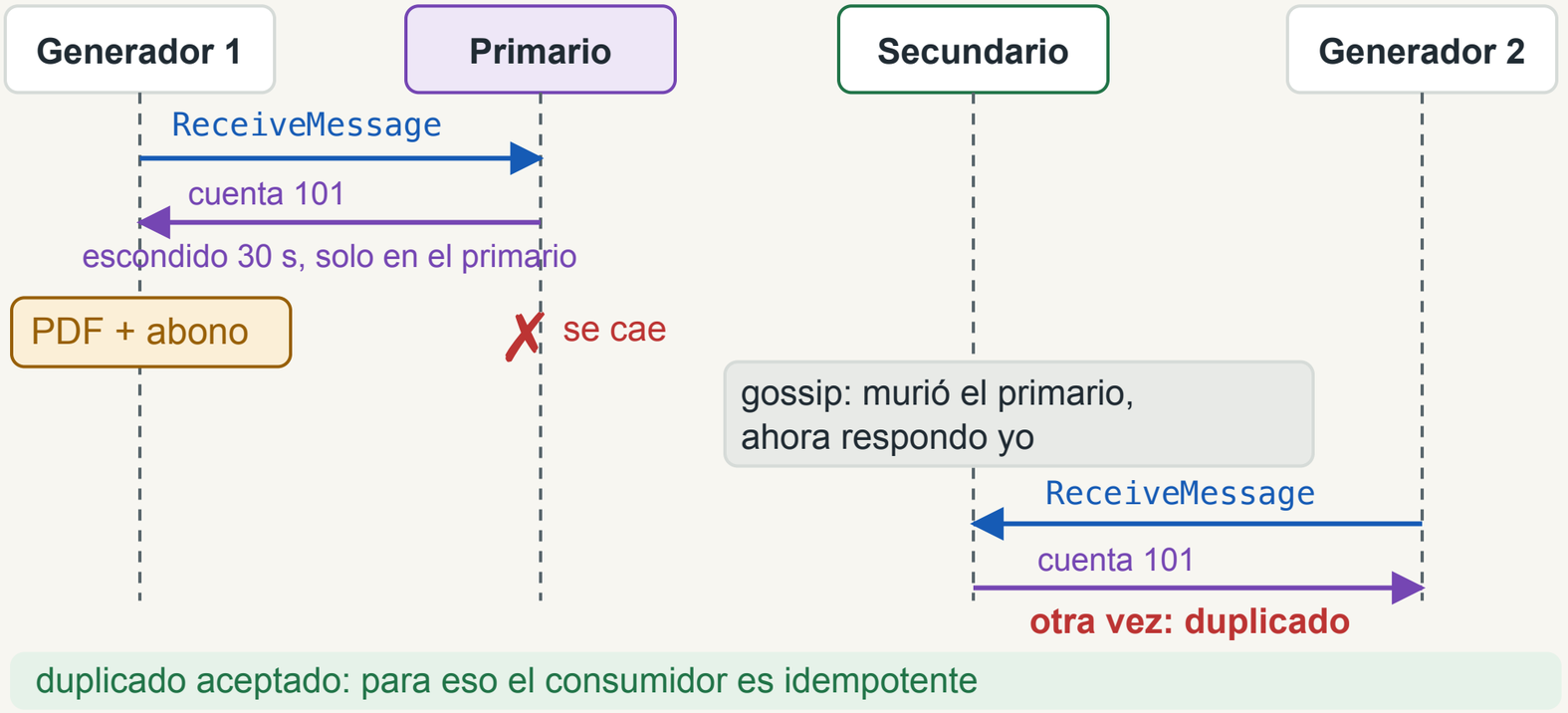
El ReceiptHandle es la dirección de regreso del mensaje, cifrada.

Réplicas y duplicados

¿Si cada mensaje se replica, por qué siguen saliendo duplicados?

Cada mensaje vive en tres nodos: primario, secundario y terciario. Solo el primario lo entrega, y solo él guarda el visibility timeout.





Los duplicados no son un bug de SQS: son el precio de no perder mensajes.

SQS y RabbitMQ

¿Y si no estás en AWS?

El patrón es el mismo con otros nombres. RabbitMQ empuja al consumidor y confirma con ack; no tiene reloj por mensaje.

Pieza	SQS	RabbitMQ
Enviar	`SendMessage`	`publish a un exchange (o sendToQueue)`
Recibir	`ReceiveMessage: el consumidor pide`	`consume: el broker empuja; prefetch limita cuántos`
Confirmar	`DeleteMessage(ReceiptHandle)`	`ack(msg)`
Consumidor caído	`vence el visibility timeout`	`se cierra el canal: vuelve a la cola`
Poison pill	`maxReceiveCount → DLQ`	`nack(msg, false, false) → DLX → DLQ`
No perder en el broker	`lo administra AWS`	`cola durable + mensaje persistent + publisher confirms`



Notificaciones ✓

Antifraude ?

Contabilidad ?

Cada mensaje llega a un solo consumidor. ¿Y si varios servicios distintos necesitan el mismo mensaje?

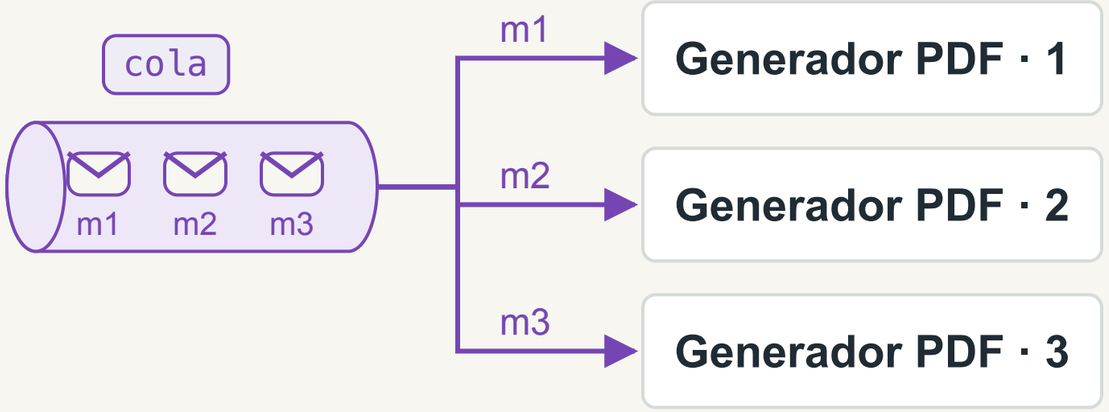
Misma cola, otro broker: entregar a uno, confirmar, reintentar y apartar lo que falla.

Consumidores heterogéneos

¿Y si tres servicios distintos necesitan el mismo mensaje?

Heterogéneos: cada uno hace algo distinto con el mismo evento. Una cola entrega cada mensaje a un solo consumidor.

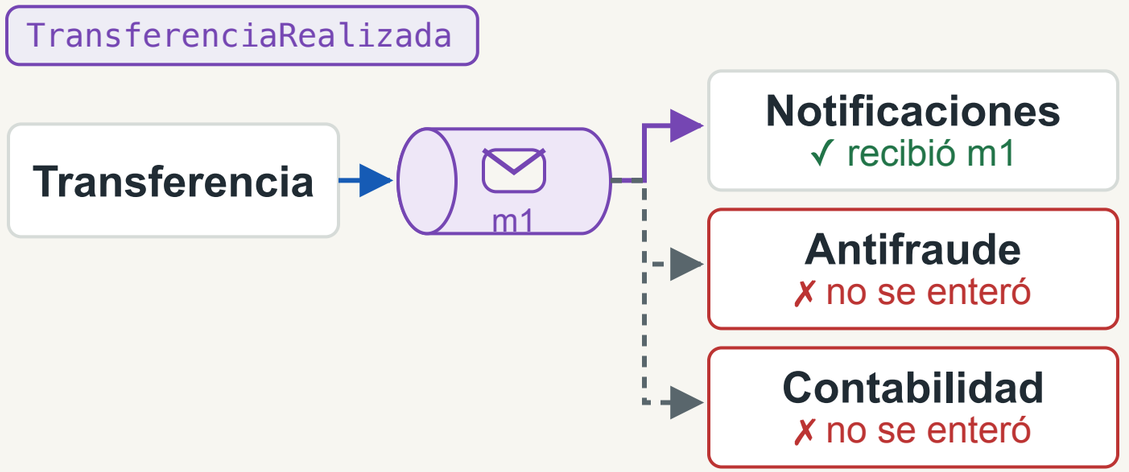
COLA CON RÉPLICAS IGUALES



✓ Cada mensaje, un solo worker

✓ Da igual cuál lo haga: balanceo

COLA CON SERVICIOS DISTINTOS



✗ Antifraude no revisa la transferencia

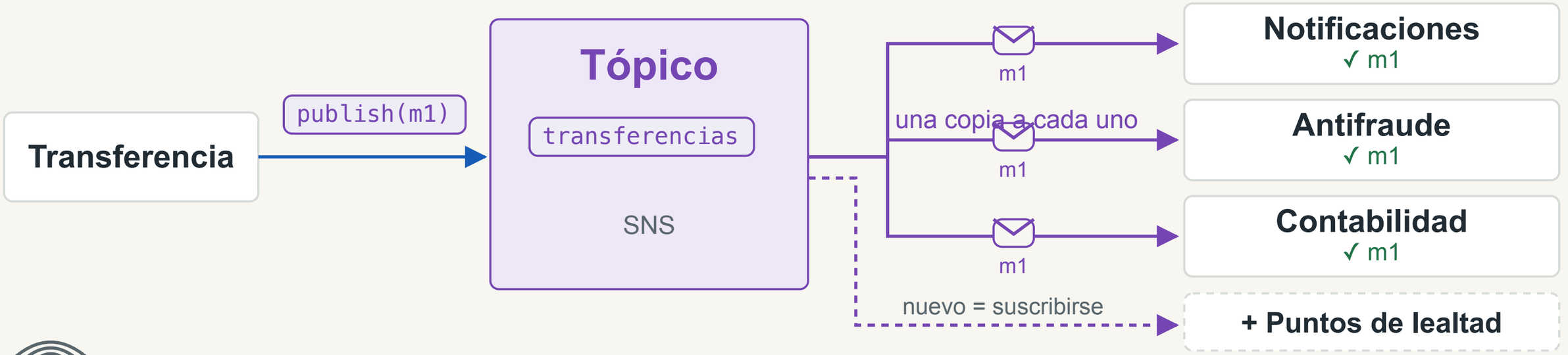
✗ Contabilidad no la registra

Una cola reparte el trabajo; aquí cada servicio necesita su propia copia.

Fan-out

¿Cómo le llega a todos el mismo mensaje?

Un tópico es un canal con nombre. Cada mensaje que se publica en él se copia a todos sus suscriptores.





Como la radio: la estación transmite una vez y la oye todo el que esté sintonizado.

`Cola: m1 → uno`

`Tópico: m1 → todos`

Se publica una vez; cada suscriptor recibe su propia copia.

Publish y subscribe

¿Cómo se usa? ¿Y dónde está el receive?

El suscriptor se registra una vez con protocolo y endpoint. El publicador solo dice tópico y mensaje; SNS entrega.

SUSCRIPTOR · UNA SOLA VEZ

```
# Antifraude, al desplegarse
sns.subscribe(
TopicArn=transferencias_arn,
Protocol="https",
Endpoint=
"https://antifraude.mexi.example/eventos",
)
```

después →

PUBLICADOR · CADA TRANSFERENCIA

```
# Transferencia, cada vez que ocurre
sns.publish(
TopicArn=transferencias_arn,
Message=json.dumps(evento),
)
```

~~`receive()`~~ No existe: SNS empuja (push) al endpoint.

⚙ **App-to-app** `sqs` `lambda` `http` `https`

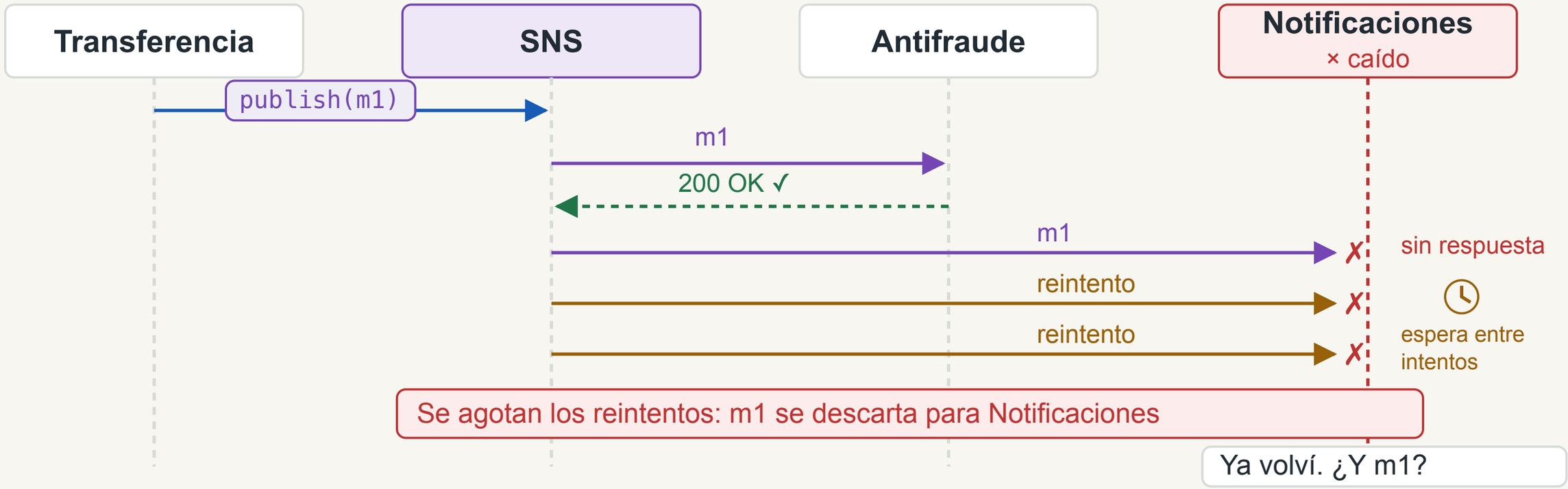
♙ **App-to-person** `email` `sms` `push móvil`

Nadie pide mensajes: SNS los empuja a cada endpoint.

Suscriptor caído

¿Qué pasa si SNS empuja y nadie contesta?

SNS reintenta la entrega unas veces; si no lo logra, descarta el mensaje para ese suscriptor. A los demás sí les llegó.

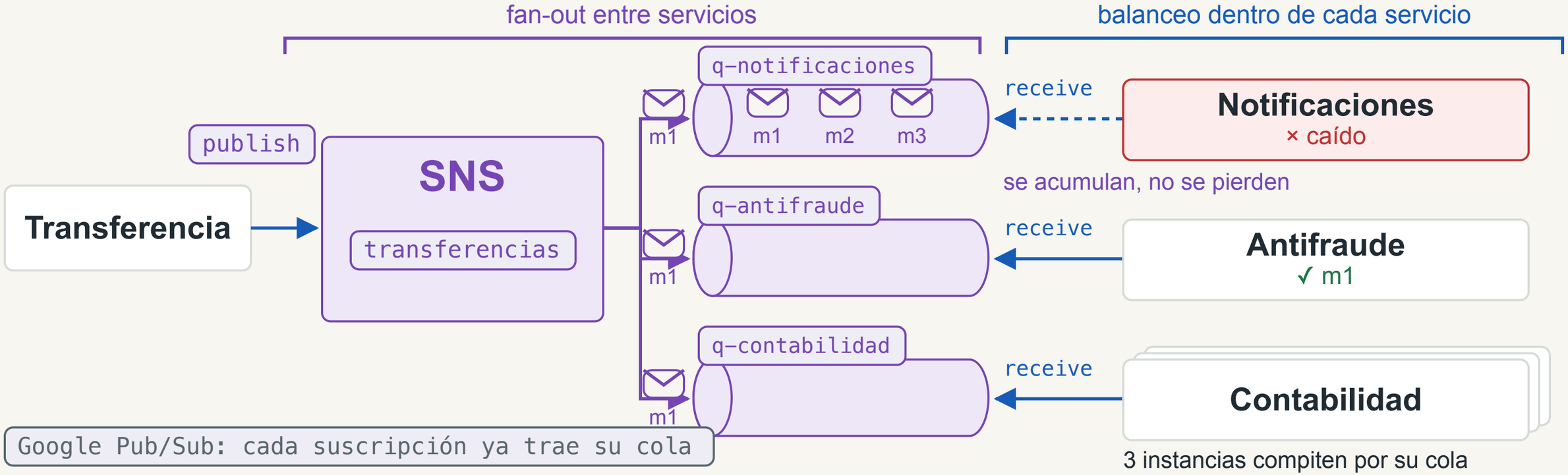


Push sin buffer: si no estabas cuando llegó, te lo perdiste.

Fan-out + buffer

¿Quién guarda el mensaje mientras el suscriptor no está?

Entre el tópico y cada servicio va una cola propia. SNS empuja a la cola; el servicio la lee cuando puede.

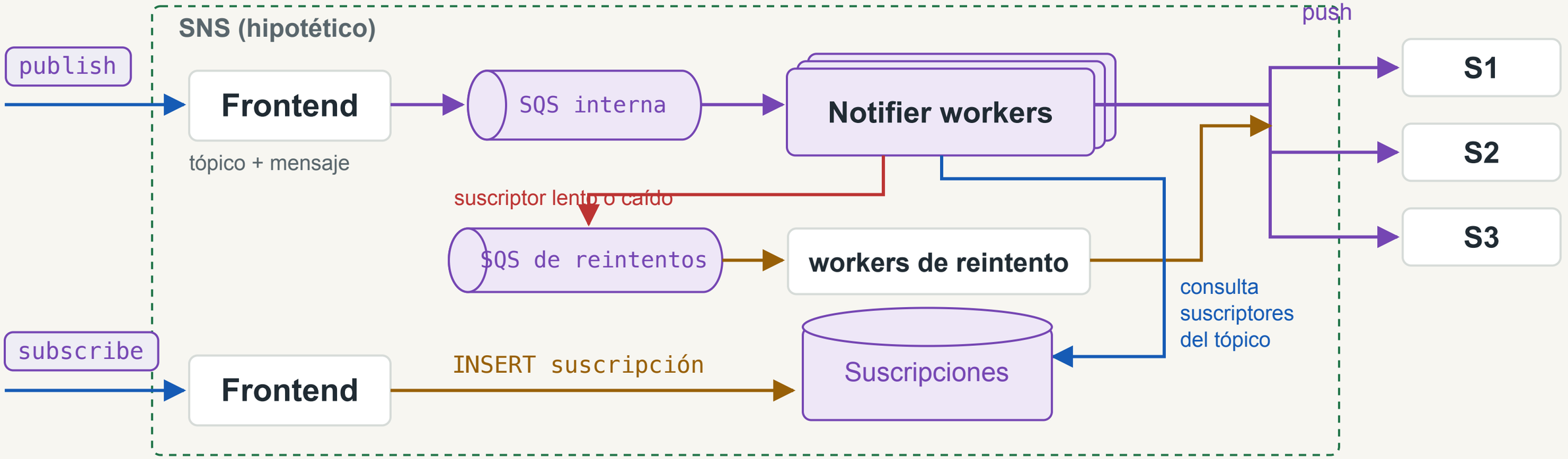


SNS reparte, cada cola guarda: cada servicio lee a su ritmo.

SNS por dentro

¿Cómo construirías un SNS con lo que ya sabemos?

Reconstrucción hipotética: AWS no la publica. El fan-out se arma con una cola interna y una flota de workers que empujan a cada suscriptor.

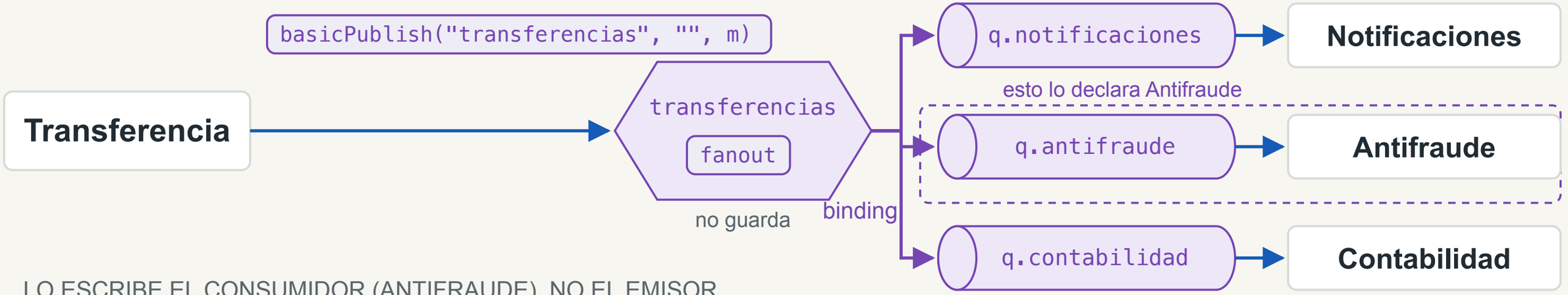


SNS es SQS por dentro: una cola de trabajo que reparte copias.

Exchange

¿Cómo se hace pub/sub en RabbitMQ?

El productor publica a un exchange; el exchange copia el mensaje a cada cola enlazada (binding). El exchange no guarda: las colas sí.



LO ESCRIBE EL CONSUMIDOR (ANTIFRAUDE), NO EL EMISOR

```
ch.exchangeDeclare("transferencias", "fanout", true);      si no existe, lo crea
ch.queueDeclare("q.antifraude", true, false, false, null); mi cola, durable
ch.queueBind("q.antifraude", "transferencias", "");        el binding = mi suscripción
ch.basicConsume("q.antifraude", false, alRecibir, alCancelar); leo con ack manual
```

En RabbitMQ el exchange copia y la cola guarda.

Topic exchange

¿Y si cada servicio solo quiere una parte?

La clave de ruteo son palabras separadas por puntos y cada cola se enlaza con un patrón:
* es exactamente una palabra, # es cero o más.

`fanout`
ignora la clave: copia a todas

`direct`
clave exacta: spei → solo spei

`topic`
patrón con * y #

Clave del mensaje	Antifraude `transferencia.#`	Contabilidad `*.realizada.*`	Conciliación SPEI `#.spei`	Notificaciones `#`
`transferencia.realizada.spei`	✓	✓	✓	✓
`transferencia.realizada.interna`	✓	✓	✗	✓
`transferencia.rechazada.spei`	✓	✗	✓	✓
`cuenta.baja`	✗	✗	✗	✓

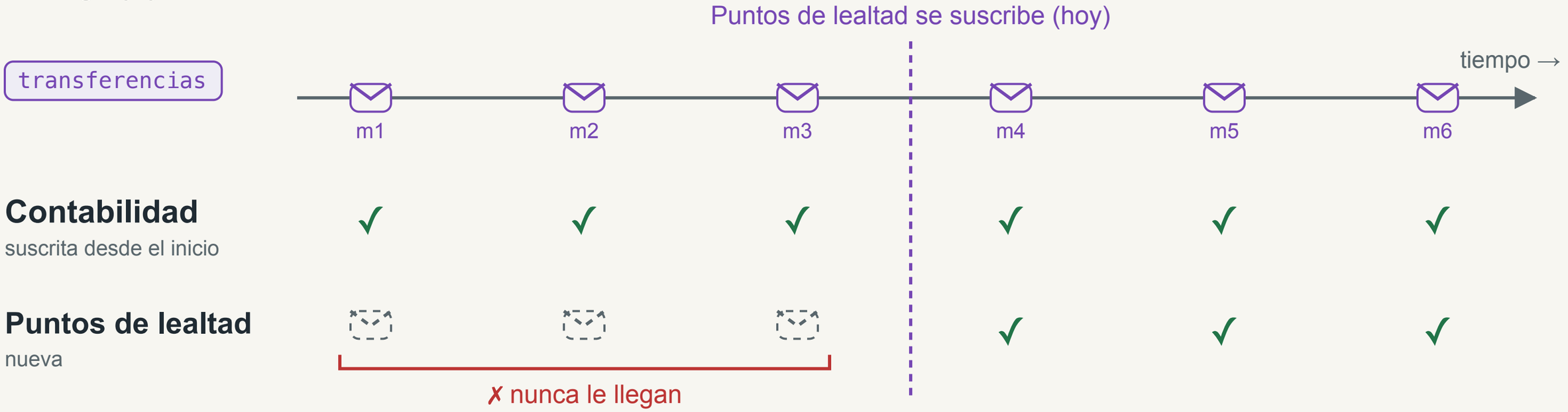
`transferencia.*` no recibe ninguna de las tres transferencias: * es una sola palabra.

El emisor pone una etiqueta; cada consumidor elige qué etiquetas oír.

Suscriptor nuevo

¿Y si quiero leer lo que pasó antes de suscribirme?

Un suscriptor recibe desde el momento en que se suscribe. Lo publicado antes ya se entregó y ya no está.



Puntos necesita el historial: ¿cuántas transferencias hizo cada cliente este año?

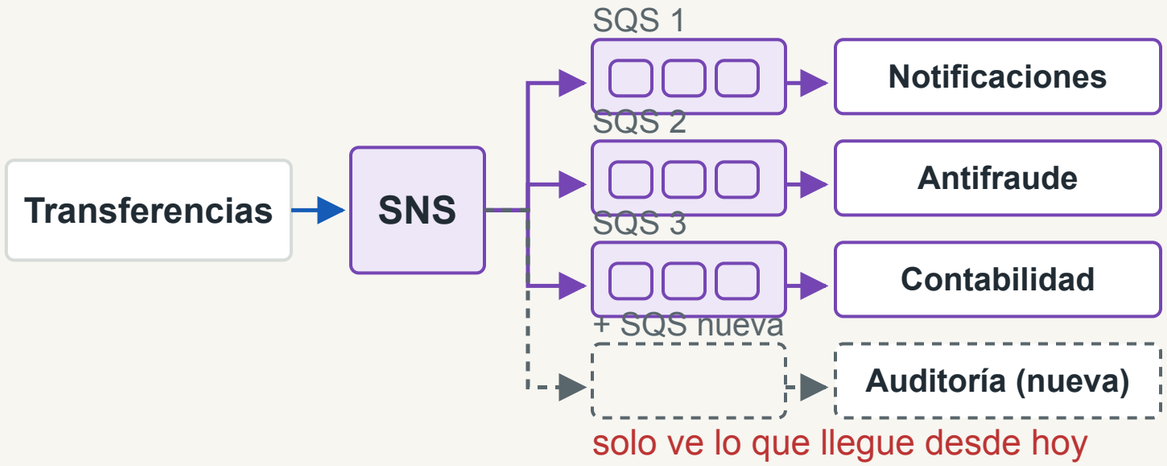
Una cola o un tópico olvidan al entregar. ¿Y si no borráramos?

Event streaming

¿Y si el evento no se puede olvidar?

Un flujo de eventos que se guardan en orden y que cada sistema lee a su ritmo, hoy o un mes después.

HOY · SNS + UNA SQS POR CONSUMIDOR



✗ Consumidor nuevo = cola nueva
✗ Al consumir se borra: no hay replay
✗ Sin orden garantizado por cuenta

LO QUE NECESITAMOS

→ **Orden**
los movimientos de una cuenta, en el orden en que pasaron

↶ **Replay**
volver a leer el último mes para reconstruir una base

\+ **Lector nuevo, sin cola nueva**
y que también pueda leer el pasado

Eventos que no son efímeros:

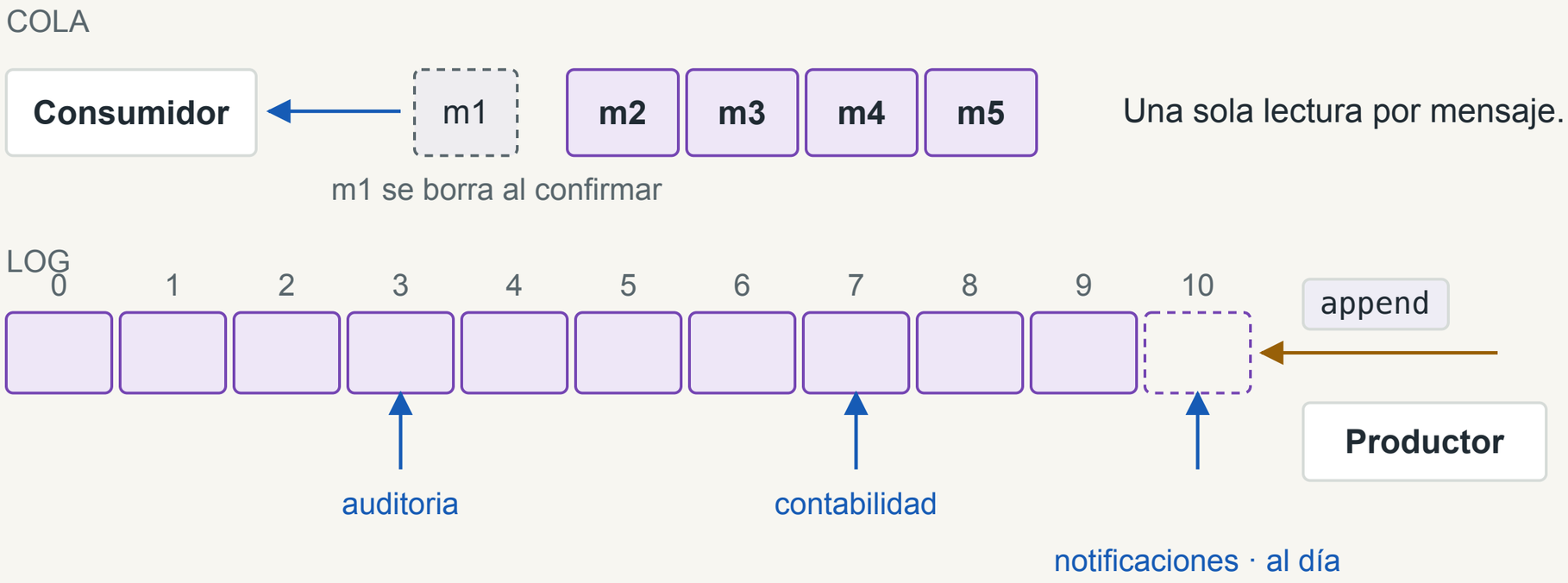
movimientos de Mexi Banco | **auditoría** | **clics de la app**

Hay eventos que son historia: se guardan, no solo se entregan.

Log

¿Y si leer no borrara?

Una secuencia que solo crece por el final (append-only). Leer no borra: cada lector solo recuerda en qué posición va.



Como un separador de libro: cada lector tiene el suyo y el libro no cambia.

En una cola, leer es consumir; en un log, leer es mover tu separador.

Topic y partición

¿Cómo escala un log sin perder el orden que importa?

Un topic se divide en N particiones; cada una es un log con sus propios offsets. El productor elige la partición con hash(key) % N.

Transferencias

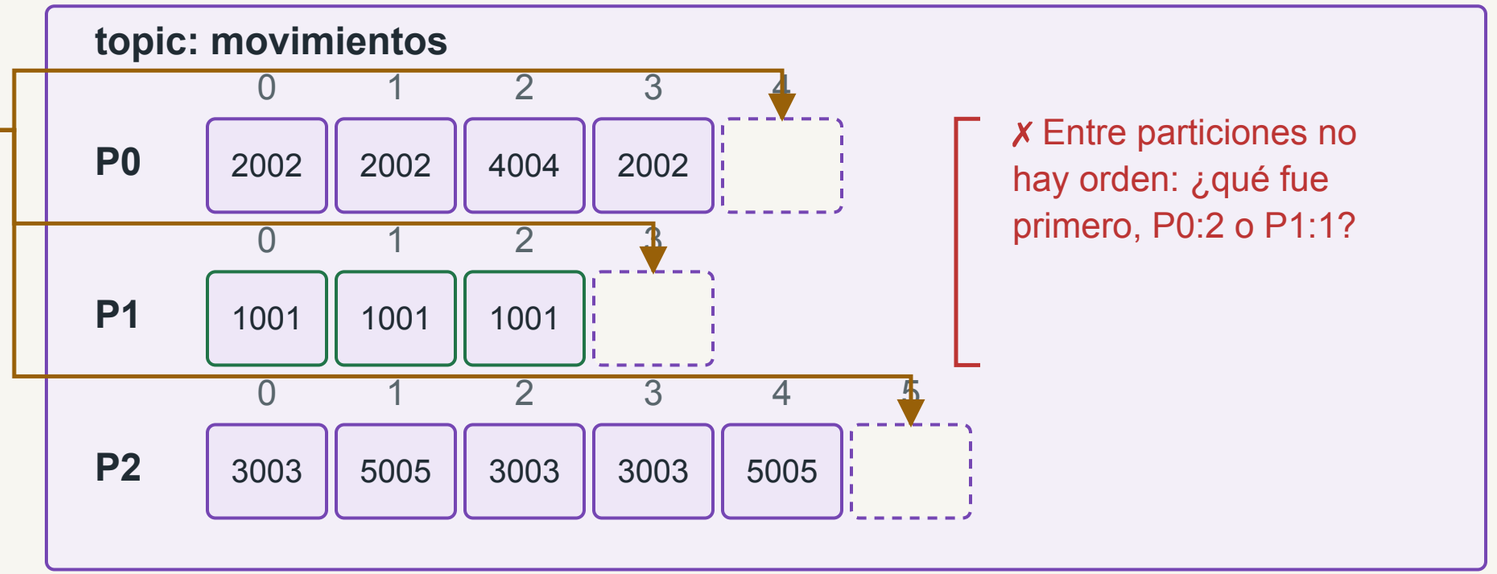
`key = cta-1001`

`key = cta-2002`

`key = cta-3003`

hash(key) % 3

mapeo ilustrativo



✓ cta-1001 → P1: movimientos en orden

Topic = el nombre del flujo

Partición = un log independiente

Offset = la posición dentro de su partición

Elige la key por lo que necesitas ordenar: en Mexi Banco, la cuenta.

API de Kafka

¿Qué escribe cada lado?

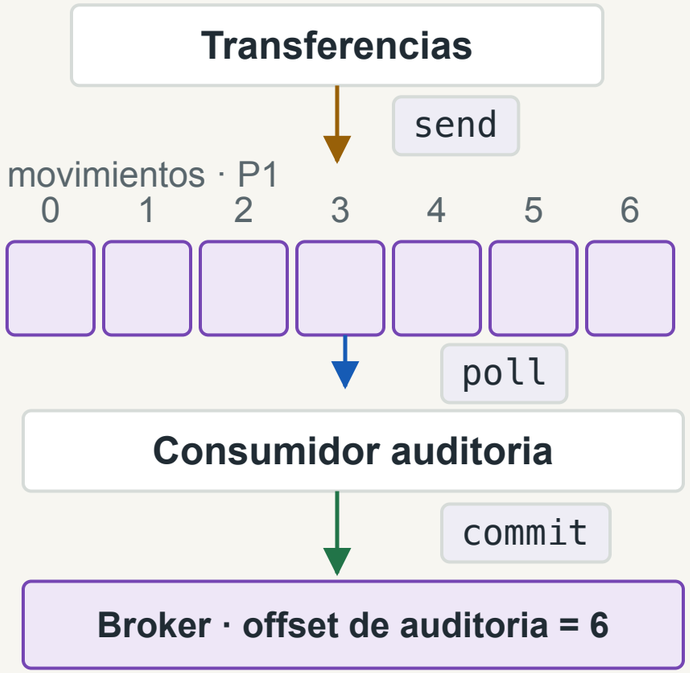
El productor envía (topic, key, valor). El consumidor se une a un grupo, pide lotes con poll y registra hasta dónde llegó con commit.

PRODUCTOR · Transferencias

```
producer.send(new ProducerRecord<>("movimientos", cuentaId, json));   1
```

CONSUMIDOR · grupo auditoria

```
props.put("group.id", "auditoria");
var consumer = new KafkaConsumer<String, String>(props);
consumer.subscribe(List.of("movimientos"));
while (true) {
for (var r : consumer.poll(Duration.ofMillis(500))) procesar(r);   2
consumer.commitSync();                                             3
}
```

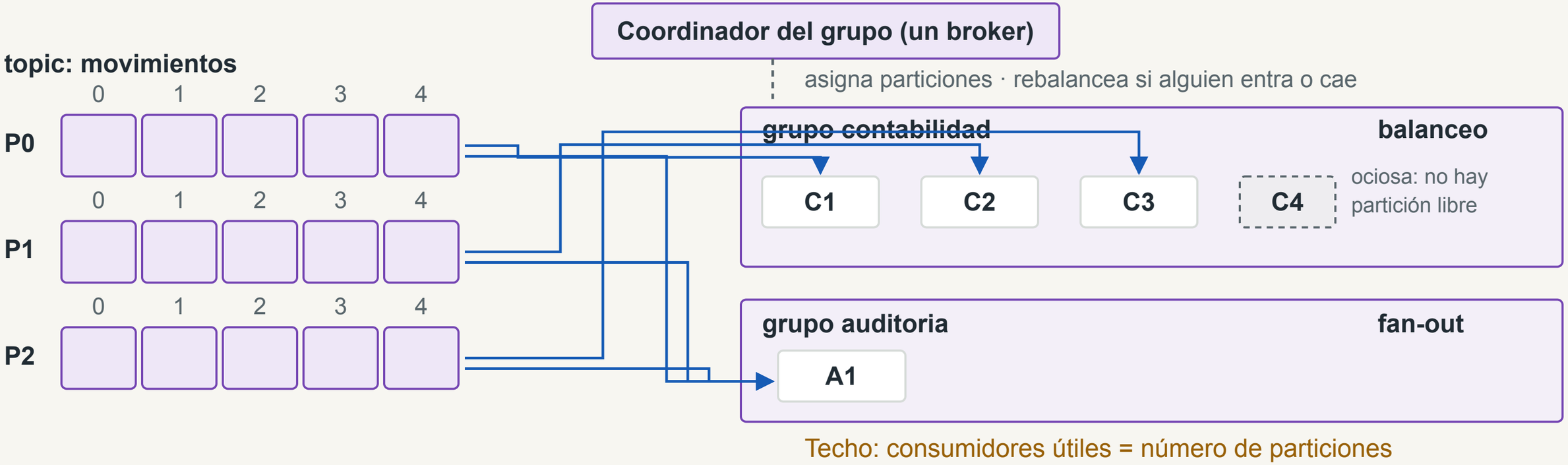


Enviar es agregar al final; consumir es leer un lote y guardar tu posición.

Consumer group

¿Cómo reparto el trabajo sin procesar todo dos veces?

Consumidores con el mismo group.id. Dentro del grupo, cada partición la lee uno solo; cada grupo distinto lee el topic completo.

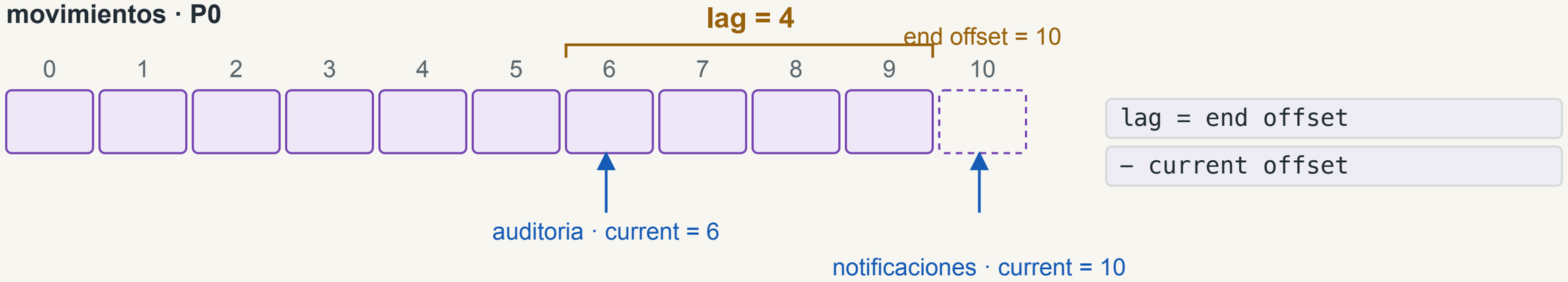


Dentro del grupo se reparte; entre grupos se copia.

Lag

¿Qué tan atrasado va cada grupo?

Lo que le falta leer a un grupo: el final del log menos su posición. La posición (current offset) es el siguiente mensaje a leer, no el último leído.




```
lag = end offset
  − current offset
```

Grupo	current offset	end offset	lag
notificaciones	10	10	0 ✓
auditoria	6	10	4

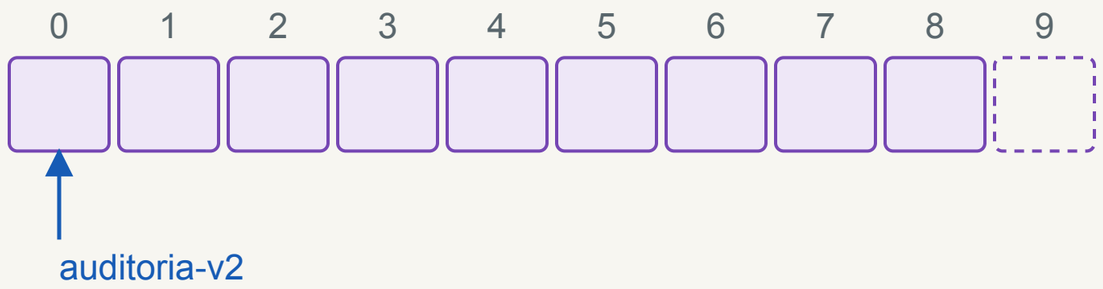
Un grupo atrasado solo se atrasa a sí mismo.

Replay

¿Y si hay que procesar todo otra vez?

Volver a leer eventos ya leídos moviendo la posición del grupo hacia atrás. Los eventos siguen en el log; solo cambia el número.

A · GRUPO NUEVO

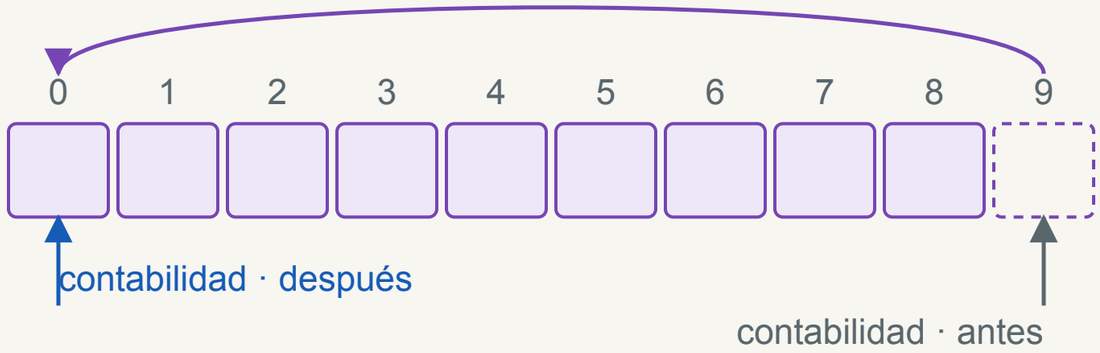



```
auto.offset.reset = earliest
```

Sin posición guardada, empieza desde el inicio.

✓ Un equipo nuevo procesa toda la historia

B · RESET DEL OFFSET




```
--reset-offsets --to-earliest
```

1 · detener el grupo (EMPTY)
2 · mover el offset
3 · volver a encender

✗ Con consumidores conectados, Kafka lo rechaza

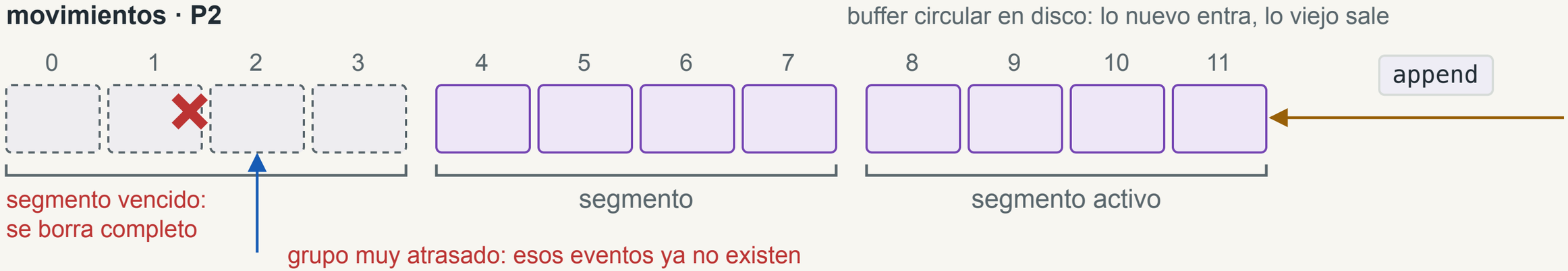
✓ Se corrigió un bug: reprocesar lo que salió mal

Rehacer el pasado es mover un número.

Retención

Si leer no borra, ¿cuándo se borra?

Kafka borra por tiempo o por tamaño, nunca por lectura. La partición se guarda en segmentos y se tira el segmento más viejo completo.



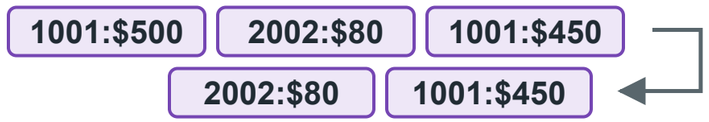
LÍMITES

`retention.ms` por tiempo: 7 días por defecto en el broker

`retention.bytes` por tamaño, medido por partición

COMPACTACIÓN `cleanup.policy = compact`

Conserva solo el último valor de cada key.

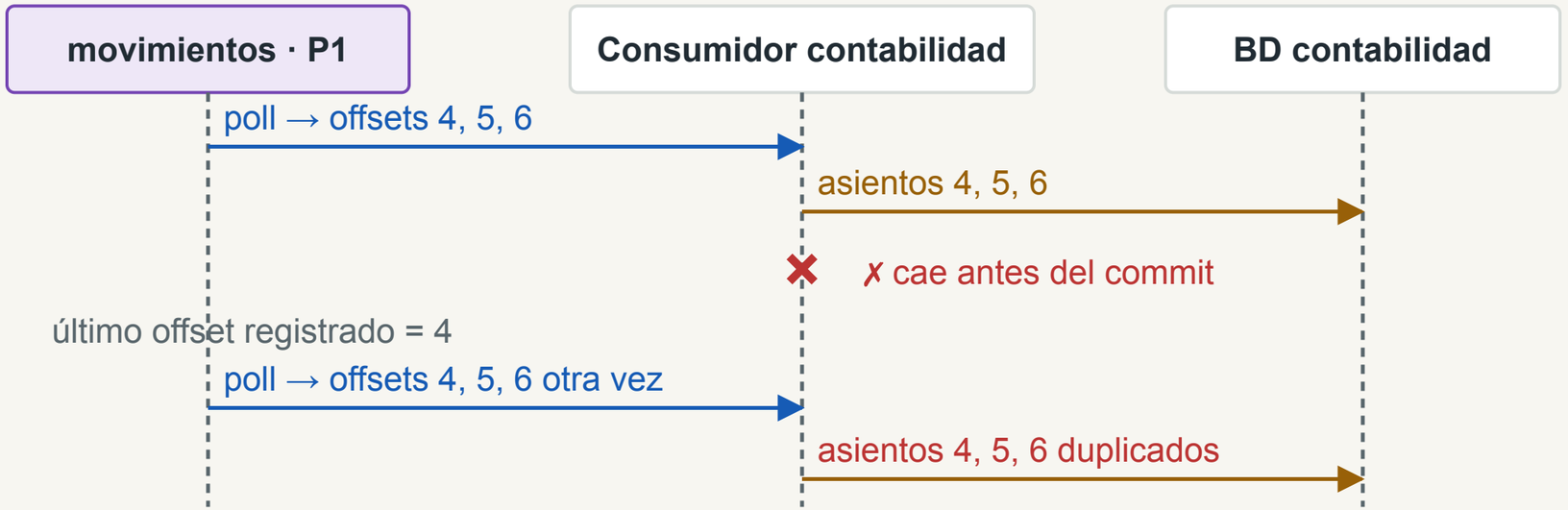


El log no es eterno: guarda una ventana, como un archivo de periódico.

At-least-once

¿Qué pasa si el consumidor cae entre procesar y hacer commit?

Al reiniciar, el grupo retoma desde el último offset registrado. Lo que se procesó después de ese commit se procesa otra vez.



IDEMPOTENCIA (S03)

```
UNIQUE (movimiento_id)
```

El segundo INSERT choca y se ignora.

Procesar dos veces equivale a procesar una.

Ya hay tres modelos: ¿cuándo cola, cuándo pub/sub y cuándo log?

Tras una caída se reprocesa: el consumidor debe ser idempotente.

Cola, tópico o log

¿Qué cambia si elijo uno u otro?

Los tres desacoplan al emisor. Cambia quién recibe cada mensaje y cuánto tiempo vive.

	Productor-consumidor `SQS`	**Publisher-subscriber** `SNS`	**Event streaming** `Kafka`
¿Quién recibe cada mensaje?	**Uno solo**	**Todos** los suscriptores	**Uno por grupo** de consumo
¿Se borra al leer?	**Sí**, con `DeleteMessage`	No lo guarda: **lo entrega**	**No**: lo borra la retención
¿Cómo se entrega?	**Pull**: `ReceiveMessage`	**Push**: SNS te llama	**Pull**: lees desde tu offset
Orden	Sin garantía	Sin garantía	**FIFO por partición**
Consumidores	**Homogéneos**: compiten	**Heterogéneos**	**Heterogéneos**, cada uno a su ritmo
¿Se puede leer el pasado?	No	No: solo lo nuevo	**Sí**: replay con el offset
Caso en Mexi Banco	Estados de cuenta en PDF, varios workers	`TransferenciaRealizada` a antifraude y contabilidad	Movimientos por CLABE, antifraude con replay

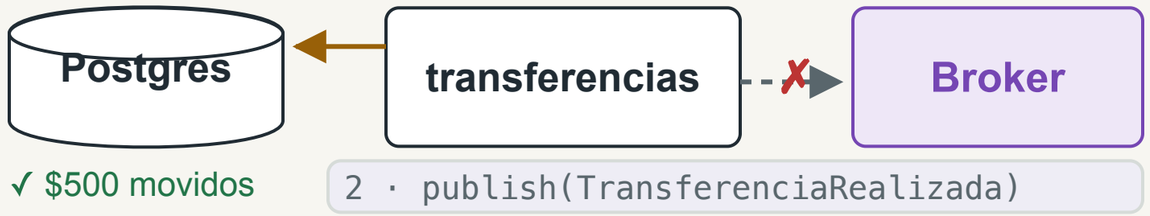
Una cola reparte el trabajo, un tópico reparte el aviso y un log guarda la historia.

Doble escritura

¿Qué pasa si el servicio cae entre guardar y avisar?

Escribir el dato en la base y el evento en el broker, en el mismo flujo, sin una transacción que los una.

GUARDAR, LUEGO PUBLICAR



La base dice que hubo transferencia.
Antifraude y notificaciones nunca se enteran.

PUBLICAR, LUEGO GUARDAR



Notificaciones avisa de $500
que nunca se movieron.

¿Y una transacción que abarque las dos escrituras?

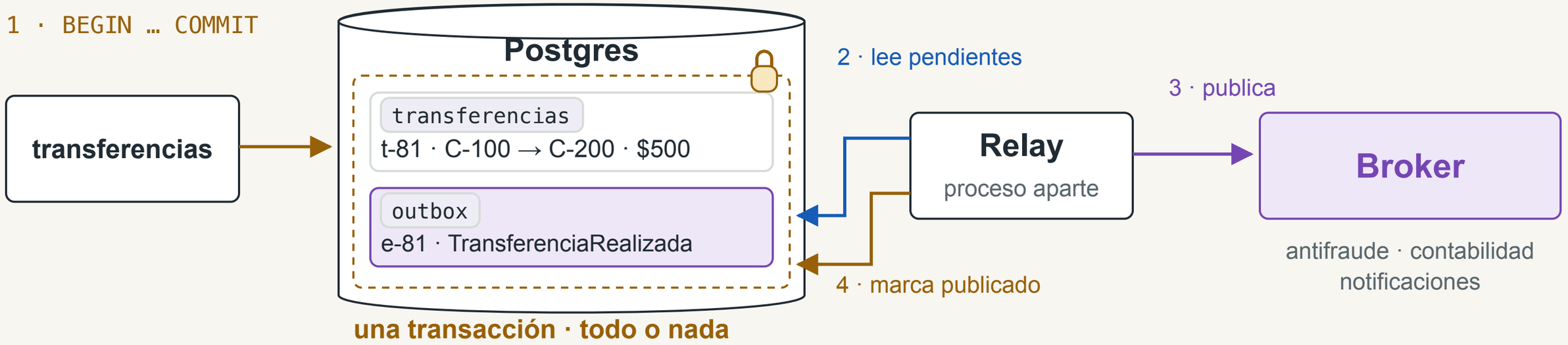
2PC existe (S08), pero RabbitMQ y Kafka no participan en él.

Dos escrituras en dos sistemas no son atómicas: tarde o temprano quedan en desacuerdo.

Outbox

¿Y si el aviso se guarda junto con el dato?

El evento se guarda como una fila de la tabla `outbox` , en la misma transacción que el dato de negocio. Otro proceso, el relay, lee esas filas y las publica después.



Como la bandeja de salida del correo:
sin red, el mensaje espera en Salida; cuando vuelve la conexión, se envía solo.

En FIUBA: «la cola como tabla».

El evento comparte el destino del dato: si uno se guarda, el otro también.

Todo o nada

¿Cómo se ve en la base?

Dos INSERT en la misma transacción local. Si algo falla antes del COMMIT, no queda ninguna de las dos filas.

```
1 BEGIN;
2 INSERT INTO transferencias (id, origen, destino, monto)
3 VALUES ('t-81', 'C-100', 'C-200', 500);
4 INSERT INTO outbox (id, tipo, payload)
5 VALUES ('e-81', 'TransferenciaRealizada',
6 '{"transferencia":"t-81","monto":500}');
7 COMMIT;
```

una sola transacción

DESPUÉS DEL COMMIT

`transferencias`

id	origen	destino	monto
t-81	C-100	C-200	500

`outbox`

id	tipo	publicado
e-81	TransferenciaRealizada	`false` pendiente

✗ Cae antes del COMMIT → ROLLBACK:
ni transferencia ni evento.

Una transacción local hace lo que no podíamos pedirle a una distribuida.

Al menos una vez

¿Y si el relay cae entre publicar y marcar?

El relay publica y luego marca. Si cae en medio, la fila sigue pendiente y el evento sale dos veces.

EL RELAY, EN BUCLE

```
1 BEGIN;
2 filas = SELECT * FROM outbox WHERE publicado = false
3 ORDER BY creado LIMIT 100 FOR UPDATE SKIP LOCKED;
4 for (fila : filas) {
5 broker.publicar(fila.id, fila.tipo, fila.payload); // espera el ack
6 UPDATE outbox SET publicado = true WHERE id = fila.id;
7 }
8 COMMIT;
```

ventana: publicado, sin marcar

- El relay publica `e-81` → broker ✓ ack
- ✗ El relay cae antes del COMMIT.
- Al volver, `e-81` sigue con `publicado = false` : lo publica otra vez.
- notificaciones recibe `e-81` dos veces: *«ya procesé e-81, lo ignoro»*

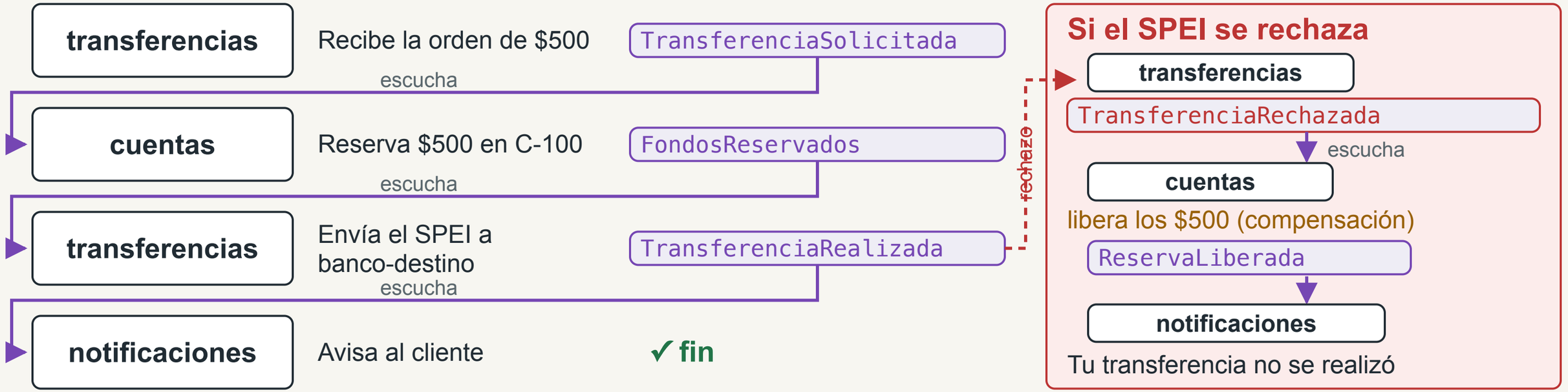
- **Varios relays:** `FOR UPDATE SKIP LOCKED` cada fila la toma uno solo (como la cola sobre Postgres).
- **Otra forma de relay:** CDC (por ejemplo, Debezium) lee el log de la base en vez de consultar la tabla.

El Outbox garantiza que el evento sale, no que salga una sola vez: el consumidor es idempotente.

Coreografía

¿Y la saga sin director?

Cada servicio escucha un evento, hace su paso local y publica el siguiente.
Nadie conoce el flujo completo; la compensación también es un evento.



En la S08, un orquestador decía quién sigue. Aquí nadie lo dice: cada uno reacciona a un hecho.

Coreografía: cada servicio escucha un hecho y publica el siguiente, cada uno con su Outbox.

Para llevar

¿Qué me llevo de hoy?

1 **Elige el patrón por cómo quieres repartir:**

a uno (cola), a todos (pub/sub), a todos y con historia (log).

2 **Asume duplicados:**

la cola, el log y el relay entregan al menos una vez. Consumidor idempotente.

3 **No escribas en dos lados sin Outbox:**

el dato y el evento, en la misma transacción.

Elegir cómo repartir, asumir duplicados y escribir una sola vez.

Decisión del día

Notificaciones de Mexi Banco: ¿cola o log?

LA DECISIÓN DEL DÍA

Notificaciones de Mexi Banco: ¿cola o log?

Cola o **Log**

Elijo: ----------

Porque: ----------

Sacrifico: ----------

Elige y di qué sacrificas: ninguna opción es gratis.